# BSCS FINAL PROJECT
## Software Design Specification

# Smart Eye Testing and Wellness Guidance

Project Advisor

**Misbah Naz**

Presented by:
**Group ID: F25CS184**


| | |
|---|---|
| L1F22BSCS0402 | Nayab Irfan |
| L1F22BSCS1019 | Hassan |
| L1F22BSCS0373 | Muhammad Ahmad |

**Faculty of Information Technology & Computer Science**

**University of Central Punjab**

# Software Design Specification

# SDP Phase II

# Smart Eye Testing and Wellness Guidance

# Advisor: Misbah Naz

# Team F25CS184

| Member Name | Primary Responsibility |
| --- | --- |
| Nayab Irfan | AI Research & Development |
| Hassan | AI & Backend Development |
| Muhammad Ahmad | Frontend UI/UX & Backend Integration |

# Table of Contents

# Revision History

| Name | Date | Reason For Changes | Version |
|---|---|---|---|
| | | | |
| | | | |

# Previous Phases Feedback

Dear Students,

Following is the feedback regarding your Idea Defence:

| | |
|---|---|
| **Sr. #** | **CS-122** |
| **Project Title** | **SMART EYE TESTING AND WELLNESS GUIDANCE** |

Similar products already exist. Well explained.

Prepare your Proposal Document in the light of comments provided. Last date to submit the Proposal Document is **Tuesday, September 16, 2025** not later than **04:00 PM**. The proposal document should be prepared using the template available at http://smrms.ucp.edu.pk/All_Phases_Templates.html. Submit the hard copy to Project Office in F-304 (Main Building).

When submitting the hard copy of the proposal document, use the duly signed Group Allocation Yellow Page (available at http://smrms.ucp.edu.pk/Forms.html) as the first page of the submission document. Also get the proposal tape bind before submission.

# Abstract

Visuar is an AI-driven vision testing and wellness guidance system designed to make eye-health screening affordable, accessible, and usable outside traditional clinical settings through a standard webcam. The system targets the widespread lack of timely eye care in resource-limited regions, where refractive conditions such as myopia, hyperopia, astigmatism, presbyopia, and emerging digital eye strain frequently go undetected, along with early behavioral signs associated with glaucoma, diabetic and hypertensive retinopathy, thyroid-related eye changes, and neurological irregularities. Visuar provides an automated and interactive screening experience by combining computer vision, eye-tracking, and lightweight machine-learning techniques to analyze ocular landmarks, blink behavior, gaze stability, color differentiation, and focus response in real time. By evaluating visual behavior and user interaction patterns, the system identifies potential vision-related risks and lifestyle-driven strain. The solution integrates concepts from deep learning, human–computer interaction, and software engineering to generate personalized wellness insights and reports. The expected outcome is a functional, non-clinical prototype capable of supporting at-home vision assessments, highlighting risk indicators that may require professional consultation, and promoting preventive eye care and early awareness of visual and systemic health concerns.

# 1. Introduction

## 1.1 Product

Visuar is an AI-based vision testing and wellness guidance software system designed to enable preliminary eye-health screening using a standard webcam and microphone. The software addresses the challenge of limited access to timely, affordable, and convenient eye care, particularly in regions where professional optometry services are scarce, costly, or difficult to access. Traditional eye examinations require physical presence, specialized equipment, and trained personnel, which often leads to delayed identification of vision problems and digitally induced eye strain. Visuar offers a non-clinical, automated, and interactive screening experience that can be performed in everyday environments such as homes, educational institutions, workplaces, and primary care settings. The end product is a research-oriented software system that applies computer vision and lightweight machine-learning techniques to analyze visual behavior and provide personalized eye-wellness guidance and informative reports.

## 1.2 Background

Advancements in artificial intelligence and computer vision have made it increasingly feasible to conduct vision-related assessments using consumer-grade devices such as webcams and personal computers. Existing mobile and desktop-based eye testing applications primarily focus on basic visual acuity or approximate refractive error estimation, often providing limited user interaction, adaptability, and personalization. In contrast, clinical AI systems employed in ophthalmology rely on high-resolution retinal imaging, specialized hardware, and controlled clinical environments, which restrict their use to professional settings and limit accessibility for the general population.[4]

Previous academic and applied research in eye tracking, blink analysis, and visual fatigue detection has demonstrated that visual behavior patterns, such as blink frequency, gaze stability, and focus response—can provide meaningful insights into eye strain and visual performance. However, many of these systems are designed for laboratory conditions or address isolated functions rather than delivering an integrated user experience. Visuar builds upon this body of work by combining multiple vision tests, real-time behavioral analysis, and wellness-oriented feedback into a single, user-friendly system.[12] The project differentiates itself by prioritizing accessibility, interactive guidance, and early awareness, while clearly positioning itself as a non-clinical screening and wellness support tool rather than a medical diagnostic solution.[8]

## 1.3 Objective(s)/Aim(s)/Target(s)

The objectives of the Visuar project are:

- To develop an AI-driven software system capable of performing preliminary eye testing and health screening using a webcam.
- To implement computer vision techniques for eye landmark detection, blink analysis, gaze tracking, and focus evaluation.
- To identify visual behavior patterns associated with eye strain and common refractive issues.
- To generate personalized eye wellness reports and recommendations based on system analysis.
- To ensure the system remains feasible, user-friendly, and achievable within the scope and duration of a BS-level final year project.

## 1.4 Scope

The scope of the Visuar project encompasses the design and development of a non-clinical eye screening and wellness guidance system based on the analysis of visual behavior using consumer-grade devices. The system focuses on identifying early indicators of digital eye strain, visual fatigue, focus instability, gaze irregularities, and approximate refractive tendencies through webcam-based interaction and user response analysis. Visuar supports automated vision testing workflows, basic user profiling, and the generation of structured eye-wellness reports intended to improve user awareness and encourage preventive eye-care practices.

The system is designed for use in non-medical environments such as homes, educational institutions, workplaces, and community centers, enabling users to perform preliminary eye assessments without professional supervision. It emphasizes accessibility, ease of use, and cost-effectiveness, making it suitable for individuals with limited access to traditional eye-care services. Visuar also supports multiple user modes, including general users, children, and elderly users, to accommodate different usability and interaction needs.

The scope of this project explicitly excludes clinical diagnosis, prescription-grade vision correction, and medical treatment recommendations. The system does not aim to replace optometrists or ophthalmologists, nor does it perform advanced ophthalmic imaging, retinal disease detection, or medical-grade analysis. Any identified risk indicators are intended solely for awareness and referral purposes. Future extensions such as integration with clinical systems, advanced diagnostic tools, or wearable medical devices are considered outside the current project scope.

## 1.5 Business Goals

The business goals of Visuar are:

- To lower the cost and accessibility barriers associated with basic eye-health screening.
- To promote early detection and preventive eye-care awareness.
- To support scalability for deployment in schools, offices, pharmacies, and rural health centers.
- To provide a foundation for future commercial or institutional health-tech solutions.
- To deliver a practical and deployable software system with real-world usability.

## 1.6 Document Conventions

This Software Requirements Specification (SRS) follows standard academic and software engineering documentation practices to ensure clarity, consistency, and ease of understanding. Section headings and subheadings are organized using a clear hierarchical structure with consistent formatting throughout the document. Bold text is used to emphasize system components, features, and important terms, while italic text is used for technical terminology, external systems, or references where appropriate.[13] Diagrams included in this document follow standard UML notation and commonly accepted modeling conventions. Terminology related to artificial intelligence, machine learning, and computer vision adheres to widely recognized definitions in computer science literature to maintain technical accuracy and consistency.[6]

## 1.7 Miscellaneous

Visuar is a research-based Final Year Project developed under academic supervision, with a strong emphasis on ethical system design, user privacy, and data security. The project adopts a modular and extensible architecture to support future enhancements and scalability beyond the current development phase. Regular supervisory evaluations, iterative documentation updates, and phased testing activities are planned to ensure structured progress and timely project completion. Appropriate disclaimers will be incorporated within the system and documentation to clearly communicate that Visuar provides preliminary eye-wellness guidance only and does not replace professional medical examination or eye care services.[14]

# 2. Overall Description

## 2.1 Product Features

Visuar provides an integrated set of AI-driven features for preliminary eye-health screening and wellness guidance. The major features of the system include:

- **At-Home Vision Screening:** Enables users to perform guided vision tests using a standard webcam without requiring specialized hardware or clinical setup.
- **Eye Tracking and Blink Analysis:** Monitors eye landmarks, blink frequency, and gaze stability to assess visual fatigue and attention patterns.
- **Visual Behavior Tests:** Includes contrast sensitivity, color differentiation, focus response, and reaction-based tests to evaluate basic visual performance.
- **Personalized Wellness Guidance:** Generates user-specific recommendations related to screen usage, rest intervals, eye exercises, and general eye care.
- **Automated Report Generation:** Produces structured, easy-to-understand eye wellness reports that can be viewed, downloaded, or shared.
- **Multimode Support:** Provides tailored interaction modes for general users, children, and elderly users to improve usability and accuracy.
- **Non-Clinical Risk Indicators:** Flags potential vision-related risks that may require professional consultation, without providing medical diagnosis.[3]


Figure 1: High-Level System Feature Overview of Visuar

## 2.2 Functional Description

Visuar operates as an interactive and intelligent software system that guides users through a structured sequence of vision assessments while continuously analyzing visual behavior in real time. The system initiates with a user interaction and environment calibration phase, during which it verifies proper face positioning, adequate lighting conditions, and webcam readiness to ensure reliable data capture. Clear on-screen and voice-based instructions assist users in maintaining correct posture and distance from the screen throughout the process.

Once calibration is complete, the webcam captures live video input of the user's face and eye region. This video stream is processed by computer vision modules that perform eye landmark detection, gaze direction estimation, blink rate measurement, and focus behavior analysis. The extracted visual features are continuously monitored and refined to handle natural head movements and variations in user behavior. Machine-learning logic then evaluates these features to identify behavioral patterns associated with digital eye strain, visual fatigue, and potential refractive irregularities.[1]

Following the analysis phase, the system aggregates the results from all vision tests and behavioral assessments. Based on this consolidated evaluation, Visuar generates a personalized eye wellness summary that includes key observations, visual performance indicators, and general guidance. The final output is presented to the user in the form of a clear, easy-to-understand report, which can be viewed on screen, downloaded, or shared for reference. Throughout the process, the system maintains a non-clinical approach, positioning itself as a preliminary screening and wellness guidance tool rather than a diagnostic medical system.[2]

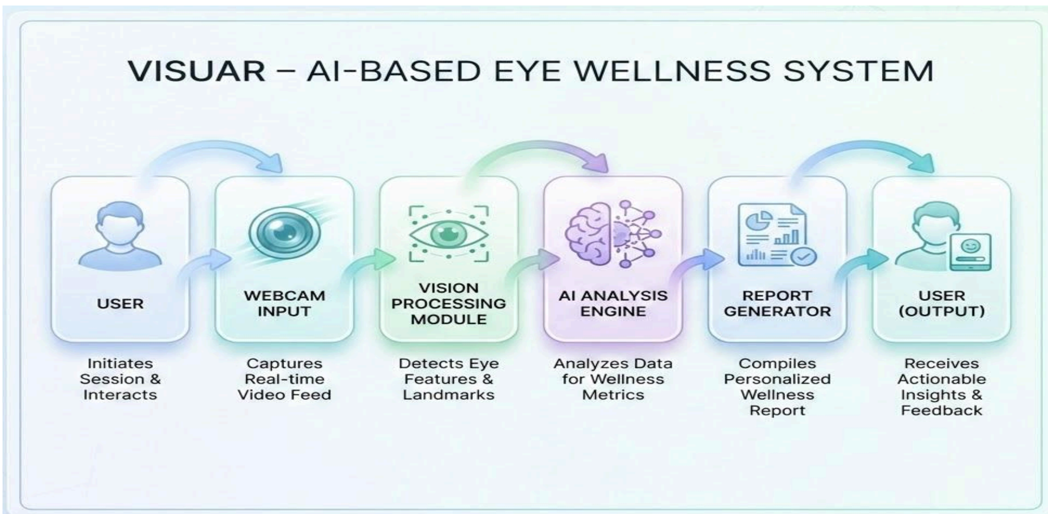


*Figure 2: Functional Description of Visuar*

## 2.3 User Classes and Characteristics

The following user classes have been identified for Visuar:

### Primary Users (High Priority)

- **General Users:** Adults seeking routine eye wellness checks. These users may have varying levels of technical literacy and require simple instructions and intuitive UI.
- **Office Workers / Students:** Users with prolonged screen exposure and higher risk of digital eye strain. The system prioritizes fatigue detection and usage guidance for this group.

### Secondary Users (Medium Priority)

- **Children:** Young users monitored under parental supervision. The system provides simplified, engaging test flows with minimal interaction complexity.
- **Elderly Users:** Individuals with age-related vision challenges such as reduced focus and presbyopia. The interface emphasizes larger text, slower pacing, and voice assistance.

## Support Users (Low Priority)

- Healthcare Support Staff / Educators: Individuals using the system for preliminary screening in schools or community settings. They require report export and repeated testing functionality.

## 2.4 Design and Implementation Constraints

The development of Visuar is subject to the following constraints:

- **Hardware Constraints:** The system must operate using consumer-grade webcams, which may vary in resolution and quality.
- **Performance Constraints:** Real-time eye tracking and analysis must run efficiently on standard laptops without requiring dedicated GPUs.
- **Non-Diagnostic Limitation:** The system must clearly remain a wellness and screening tool, not a medical diagnostic device.
- **Technology Constraints:** Development is limited to approved tools and frameworks such as Python, OpenCV, MediaPipe, and FastAPI.
- **Privacy and Security Constraints:** Webcam data must not be stored without explicit user consent, and user information must be securely handled.
- **Time Constraints:** The scope and complexity must remain achievable within the Final Year Project timeline.

## 2.5 Assumptions and Dependencies

### Assumptions

- Users have access to a functional webcam and basic lighting conditions.
- Users follow on-screen and voice instructions during testing.
- The system is used for preliminary screening and wellness guidance only.
- Internet connectivity is available when required for optional services.

### Dependencies

- Dependence on third-party libraries such as OpenCV and MediaPipe for vision processing.
- Availability of machine-learning models compatible with real-time execution.
- Operating system support for webcam and microphone access.
- Browser or desktop permissions allowing camera usage.[1]

# 3. Technical Architecture

Visuar is a **custom-built, research-oriented software system** designed specifically for webcam-based eye screening and wellness guidance. The system is not based on any off-the-shelf (COTS) eye-testing solution; instead, it integrates open-source computer vision libraries with custom probabilistic inference logic to achieve its objectives.

The system primarily performs **online, real-time processing**. All vision tests, eye tracking, behavioral analysis, and probabilistic inference are executed live during user interaction. In addition to online processing, the system also performs **analytical reporting**, where session data is aggregated, analyzed, and stored in structured formats for post-session review.

The architecture follows a **modular, layered design**, where data flows sequentially from input acquisition to inference and reporting. Each module is loosely coupled, allowing individual components to be modified or extended without affecting the rest of the system.

## 3.1 Application and Data Architecture

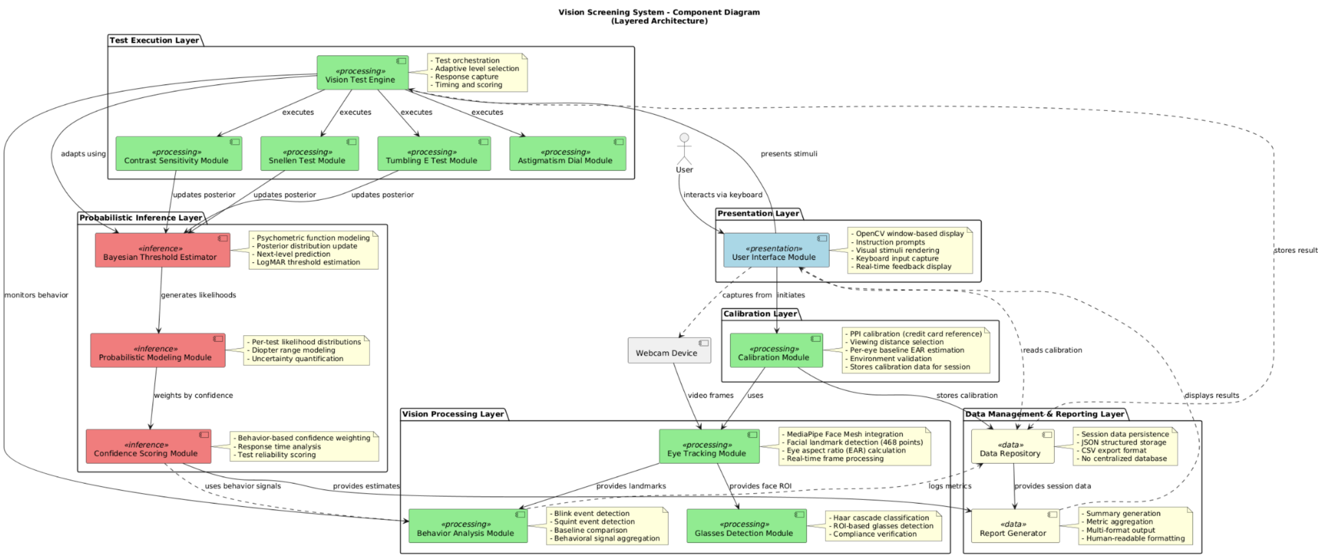


*Figure 3: High-Level Application Architecture Diagram*

### 3.1.1 High-Level Application Architecture

At a high level, the system is organized into the following logical layers:

1. **Presentation Layer**
2. **Vision Processing Layer**
3. **Test Execution Layer**
4. **Probabilistic Inference Layer**
5. **Data Management and Reporting Layer**

This layered structure ensures separation of concerns, improves maintainability, and supports future extensibility.

### 3.1.2 Application Components

#### 1. Presentation and Interaction Module

This component is responsible for all user interaction and guidance during the testing session. It displays instructions, calibration prompts, visual stimuli, and feedback messages. User input is captured via keyboard interactions.

Although the current prototype uses a desktop-based interface rendered through OpenCV windows, the architecture is UI-agnostic and can later support web or mobile interfaces.[11]

#### 2. Calibration Module

The calibration module ensures measurement reliability by normalizing environmental and user-specific variables. It performs:

- Screen pixel density (PPI) calibration using a physical reference object
- Viewing distance selection
- Per-eye baseline eye aspect ratio (EAR) estimation

Calibration data is stored and reused across all subsequent vision tests within a session.[5]

#### 3. Eye Tracking and Behavior Analysis Module

This module continuously processes live webcam video frames to detect facial landmarks and extract eye-related metrics. Using eye aspect ratio (EAR), it detects:

- Blink events
- Squint events
- Average eyelid openness

Behavioral signals are collected throughout the session and later used to modulate test confidence and fatigue estimation.

#### 4. Vision Test Engine

The vision test engine orchestrates the execution of multiple vision tests. Each test is implemented as an independent unit but follows a common interface. The system currently includes seven tests:

- Snellen Acuity
- Contrast Sensitivity
- Orientation Discrimination (Tumbling E)
- Rapid Recognition
- Sustained Focus
- Blur Tolerance
- Near-Far Switching

Each test produces structured outputs including raw scores, accuracy, response timing, and associated behavioral metrics.

## 5. Probabilistic Modeling Module

This module converts raw test outcomes into probabilistic representations. Instead of producing single deterministic scores, each test generates a **likelihood distribution** over a discrete diopter range. These likelihoods model uncertainty and variability inherent in human vision testing.

Each test has its own probabilistic model, tailored to the nature of the test and its relationship with refractive error or fatigue.

### 6. Bayesian Fusion Engine

The Bayesian Fusion Engine combines likelihood distributions from all tests into a unified posterior distribution per eye. Test contributions are weighted using confidence scores derived from behavioral indicators such as blink rate, squint frequency, and response time.

The output of this module includes:

- Mean diopter estimate
- Standard deviation
- Confidence interval
- Fatigue score
- Refractive indicators

## 7. Data Management and Reporting Module

This module is responsible for persisting session data and generating reports. All results are stored locally in:

- **JSON format** for structured analysis
- **CSV format** for human-readable summaries

No centralized database is used in the current prototype, aligning with the system's offline-first and privacy-conscious design.[2]

## 3.1.3 Data Architecture

The system manages the following primary data entities:

- **Calibration Data**: PPI, viewing distance, baseline EAR values
- **TestResult**: Per-test results including raw scores, accuracy, and behavior
- **BehaviorMetrics**: Blink count, squint count, response times
- **DiopterLikelihood**: Probability distribution over refractive power
- **EyeEstimate**: Final per-eye inference results
- **SessionData**: Aggregated session-level information

Data relationships follow a hierarchical structure where a session contains multiple test results, which in turn contain behavioral metrics.

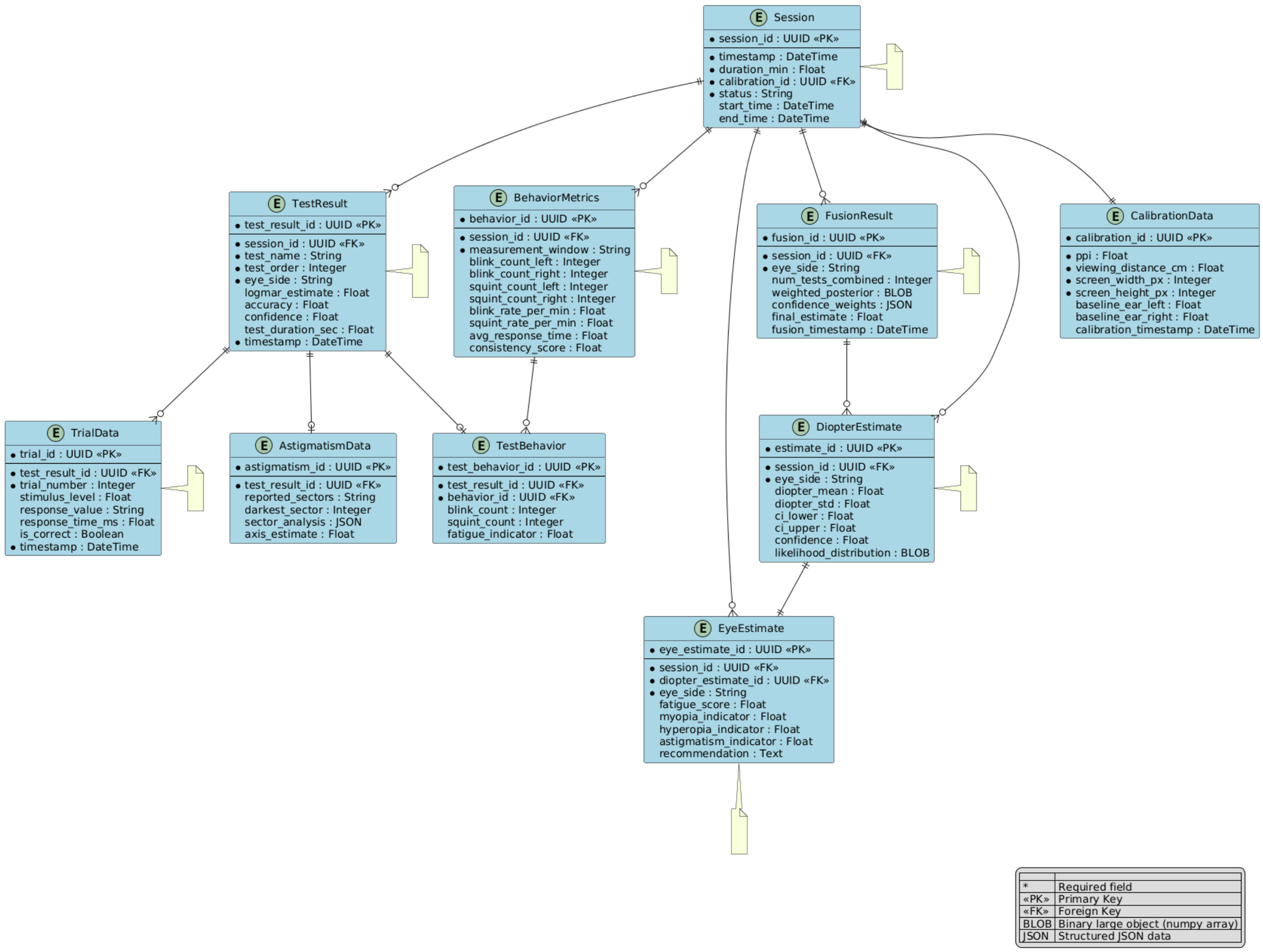


*Figure 4: System Data Entities and Relationships*

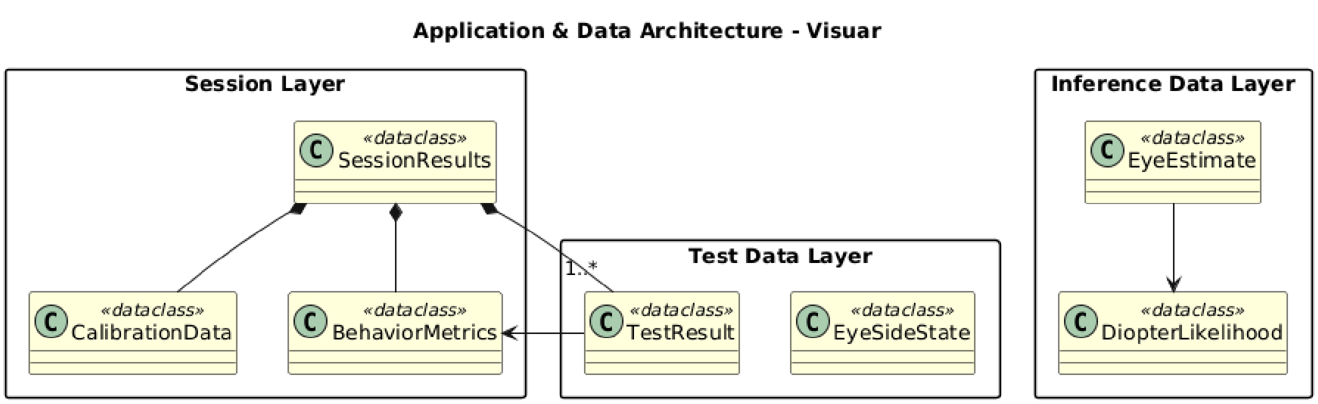


*Figure 5: Suggestion based on Content: Detailed View of a Core Component*

*Visuar*

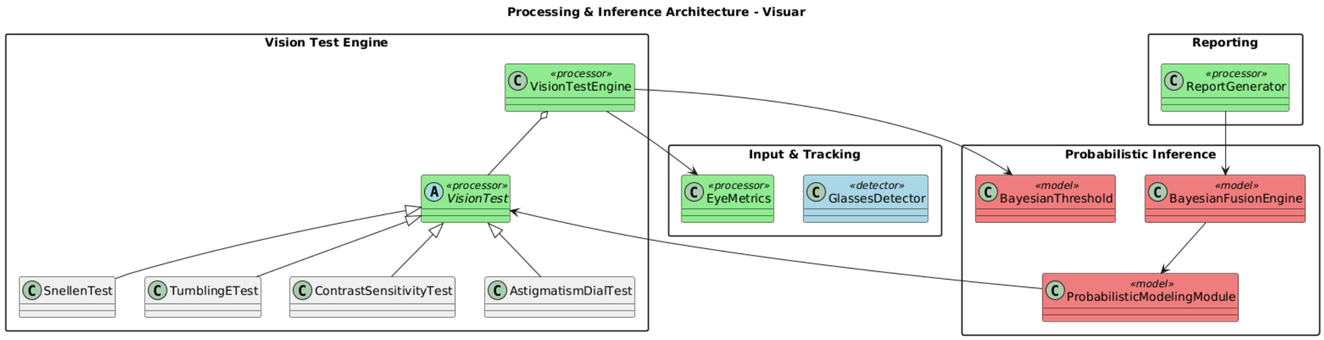


*Figure 6: Data Flow Diagram (DFD) of the Visuar System*

At the core of the system is the **Vision Test Engine**, which orchestrates the execution of multiple vision tests, including Snellen, Tumbling-E, Contrast Sensitivity, and Astigmatism Dial tests. Each test module is responsible for generating visual stimuli, collecting user responses, and forwarding trial outcomes to the central engine. This modular design allows different tests to operate independently while sharing a common execution and control flow.

In parallel with test execution, the **Input and Tracking** subsystem continuously monitors user behavior through eye metrics and glasses detection. EyeMetrics extracts blink and squint indicators, while the Glasses Detector ensures testing conditions remain valid. These behavioral signals are fed back to the Vision Test Engine and later used to adjust confidence estimates and fatigue indicators, without interrupting the test flow.

Test outcomes from the Vision Test Engine are passed to the **Probabilistic Inference** layer, which forms the analytical core of Visuar. The Bayesian Threshold component models user responses under uncertainty, while the Probabilistic Modeling Module converts individual test results into likelihood distributions. These likelihoods are then combined by the Bayesian Fusion Engine to produce a consolidated posterior estimate that reflects overall visual performance along with associated uncertainty.

Finally, the inferred results are forwarded to the **Reporting** component, where the Report Generator formats outputs into structured reports suitable for both human interpretation and machine processing. This includes numerical estimates, confidence measures, and behavioral summaries. The overall data flow emphasizes separation of concerns, explainability, and robustness, ensuring that Visuar operates as a non-clinical, decision-support system rather than a black-box predictor.

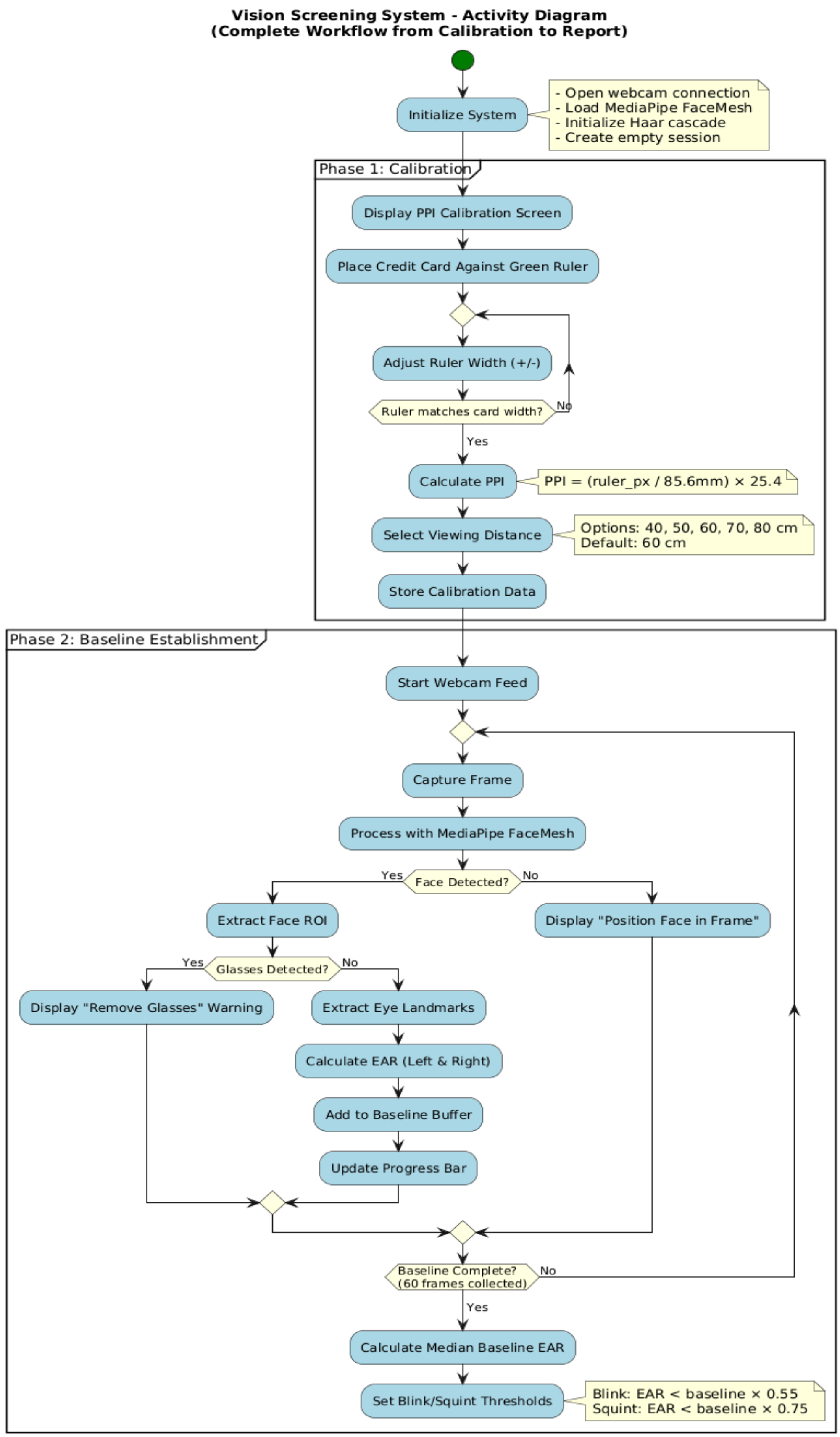


*Figure 7: User Interface Mockup or Wireframe*

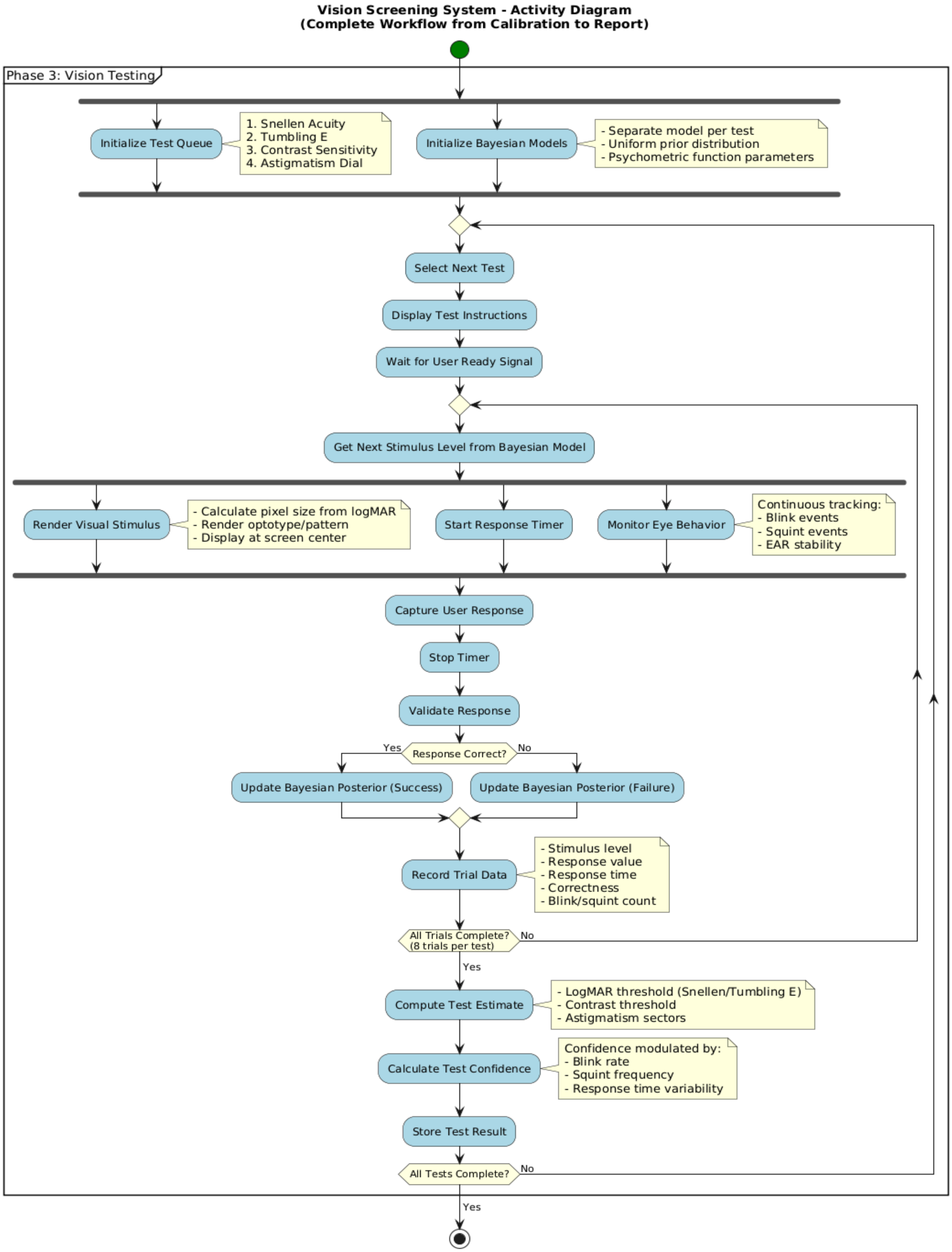


*Figure 8: Vision Processing Pipeline*

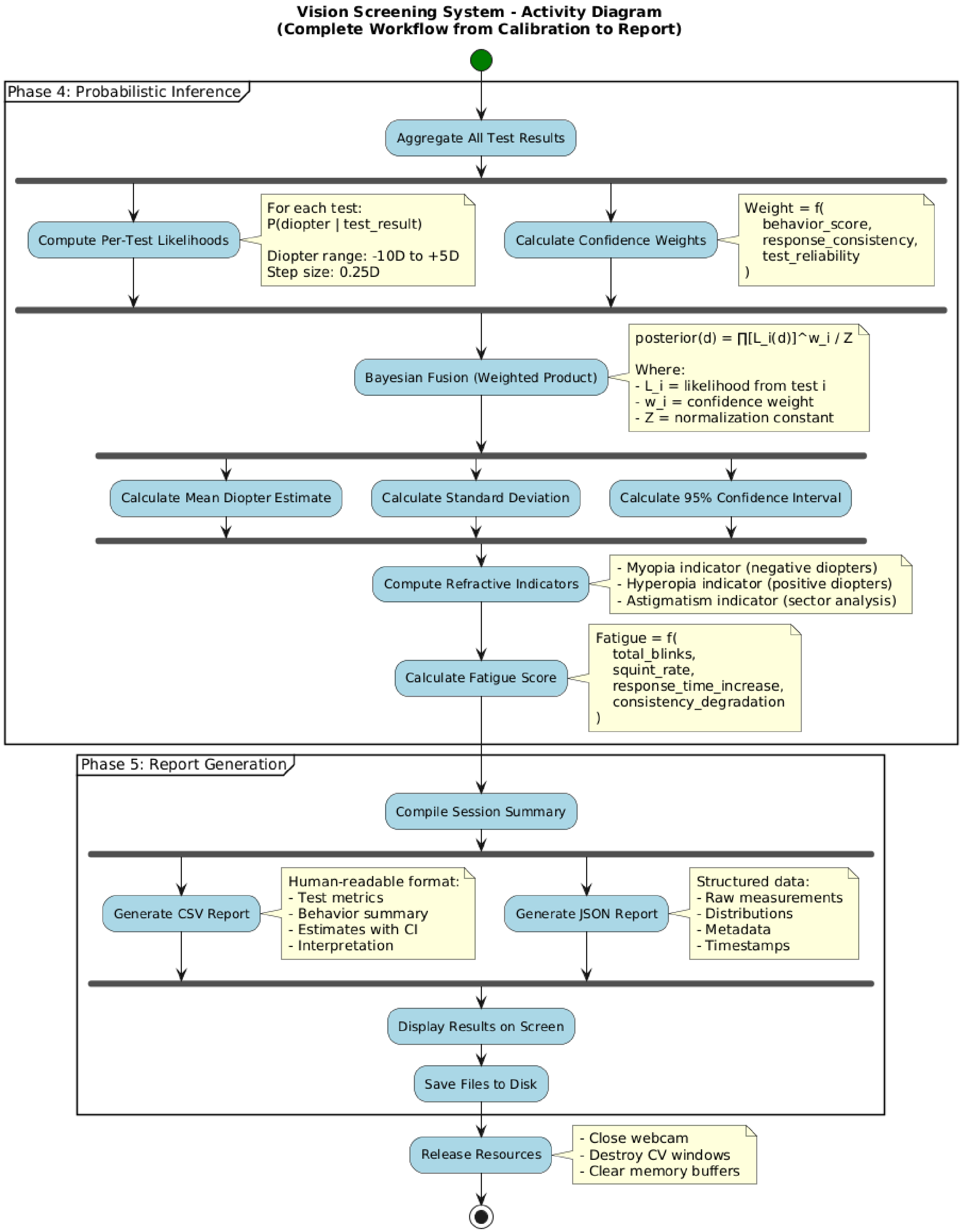


*Figure 9: Probabilistic Modeling Workflow*

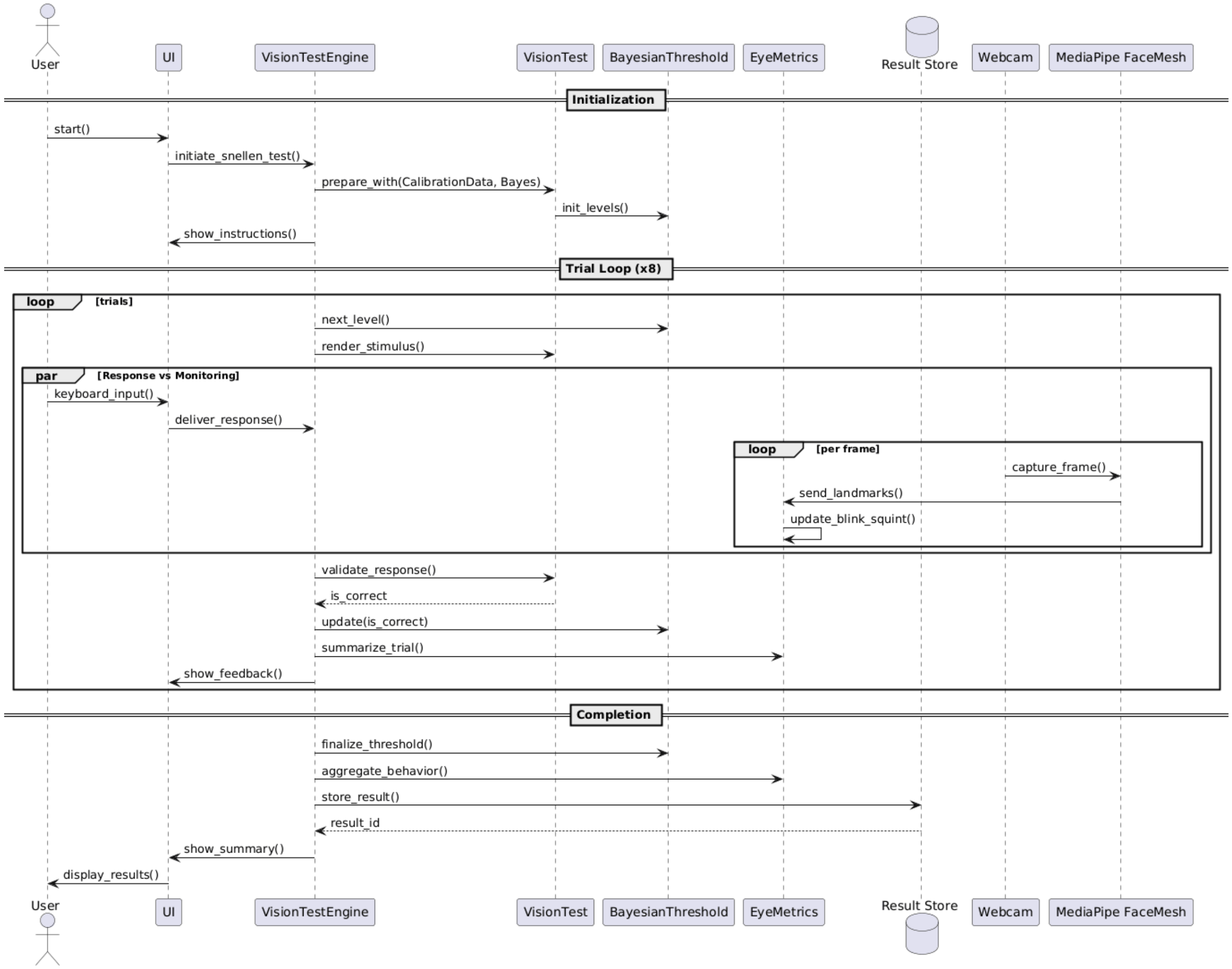


*Figure 9: initial phase trial loop*

The sequence begins with the **Initialization phase**, where the user starts the test through the UI. The UI delegates control to the Vision Test Engine, which prepares the selected vision test using previously computed calibration data and initializes the Bayesian Threshold model. At this stage, test parameters and difficulty levels are configured, and the user is presented with on-screen instructions to ensure consistent test conditions before execution begins.

During the **Trial Loop**, which is repeated a fixed number of times, the Vision Test Engine coordinates adaptive testing behavior. For each trial, the Bayesian Threshold model selects the next stimulus level based on the current belief state, and the Vision Test module renders the corresponding visual stimulus. The UI then displays the stimulus to the user. Two activities occur in parallel during each trial: user response capture and continuous behavior monitoring. While the user provides input via the keyboard, the EyeMetrics module simultaneously processes video frames obtained from the webcam through MediaPipe FaceMesh to track blink and squint behavior on a per-frame basis.

Once a response is received or the trial window expires, the Vision Test Engine validates the response and determines correctness. The Bayesian Threshold model is then updated using the trial outcome, allowing the system to iteratively refine its estimate of the user's visual threshold. Behavioral data collected during the trial is summarized and associated with the trial outcome, supporting later confidence and fatigue analysis. Immediate feedback is displayed to the user after each trial to maintain engagement and clarity.

In the **Completion phase**, the Vision Test Engine finalizes the threshold estimate after all trials are completed and aggregates behavioral metrics collected throughout the test session. The complete test result, including performance metrics and behavioral summaries, is stored in the result repository. Finally, the UI retrieves the stored result and presents a consolidated summary to the user. Overall, this sequence diagram demonstrates how Visuar integrates adaptive probabilistic inference with real-time behavioral monitoring in a coordinated and explainable execution flow.

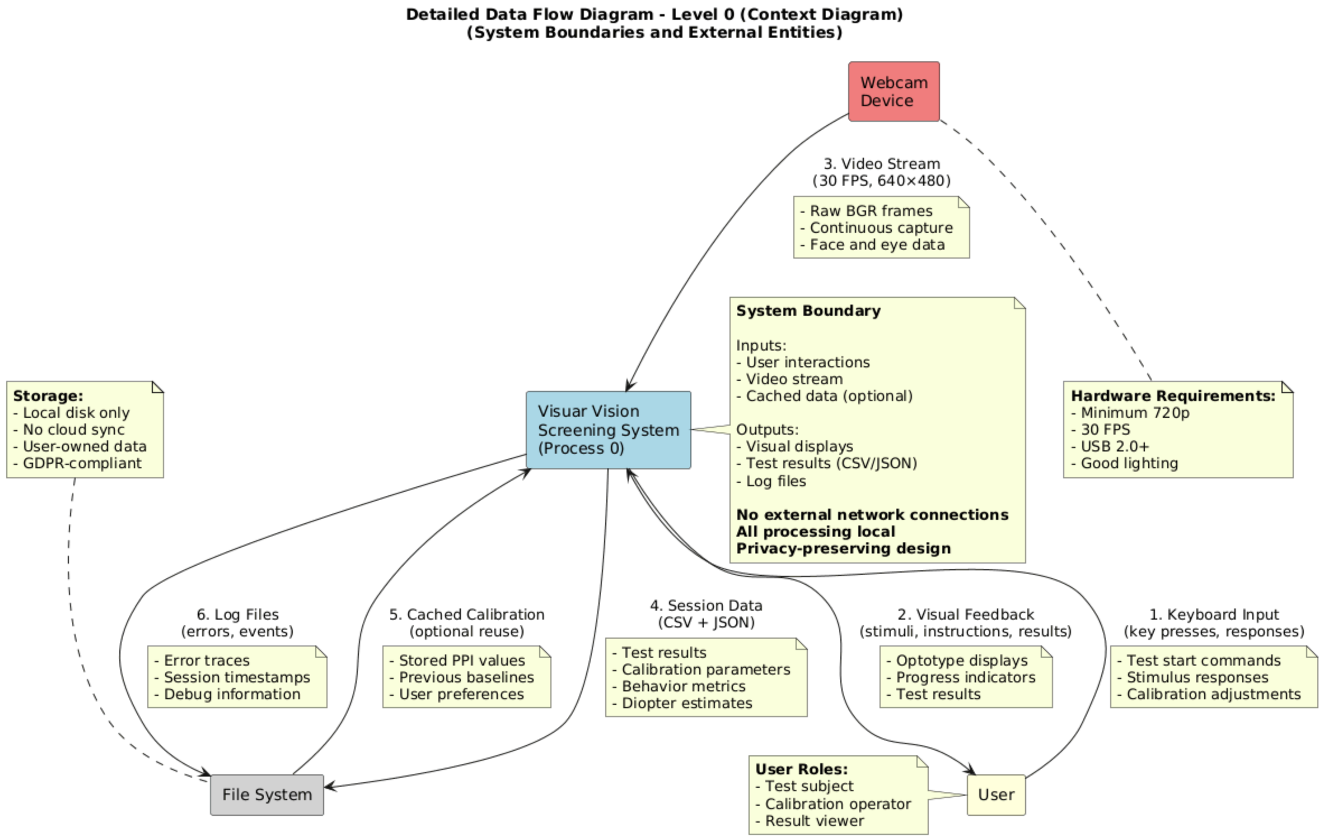


*Figure 10:* System Deployment Diagram

This Level-0 Data Flow Diagram represents the Visuar Vision Screening System as a single high-level process and defines its system boundaries, external entities, and primary data exchanges. The diagram shows how the user interacts with the system through keyboard inputs for calibration adjustments, test responses, and navigation, while receiving visual feedback such as stimuli, progress indicators, and result summaries.

The webcam device provides a continuous video stream that supplies raw image data for face and eye analysis, enabling behavior tracking during testing. All processing is performed locally within the system boundary, and no external network connections are required, ensuring privacy-preserving operation. The local file system serves as persistent storage for cached calibration data, session results, and log files used for debugging and traceability. This diagram provides a high-level overview of Visuar's operational environment, emphasizing offline execution, controlled data flow, and clear separation between the system and its external interfaces.

### 3.1.4 Decision Table

## Test Confidence Scoring Decision Table

| Rule # | Blink Rate (per min) | Squint Count | Response Time Variability | Response Consistency | **Confidence Score** | **Interpretation** |
|---|---|---|---|---|---|---|
| 1 | < 15 | 0-2 | Low (CV < 0.3) | High (> 75% correct) | 0.85 - 1.00 | Very High |
| 2 | 15-25 | 3-5 | Moderate (CV 0.3-0.5) | High (> 75% correct) | 0.70 - 0.85 | High |
| 3 | 15-25 | 6-10 | Moderate (CV 0.3-0.5) | Medium (50-75% correct) | 0.55 - 0.70 | Medium |
| 4 | > 25 | 6-10 | High (CV > 0.5) | Medium (50-75% correct) | 0.40 - 0.55 | Low-Medium |
| 5 | > 25 | > 10 | High (CV > 0.5) | Low (< 50% correct) | 0.20 - 0.40 | Low |
| 6 | > 40 | > 15 | Very High (CV > 0.8) | Very Low (< 30% correct) | 0.00 - 0.20 | Very Low |

*Figure 11: Test Confidence Scoring Decision Table*

**Confidence Calculation Formula:**
Confidence = w1 × blink_score + w2 × squint_score + w3 × rt_score + w4 × consistency_score
Where:
1 - blink_score = max(0, 1 - (blink_rate / 40)) - squint_score = max(0, 1 - (squint_count / 20)) - rt_score = max(0, 1 - (rt_cv / 1.0)) - consistency_score = accuracy - Weights: w1=0.25, w2=0.30, w3=0.20, w4=0.25

## Fatigue Level Classification Decision Table

Purpose: Classify user fatigue level based on aggregated session behavior metrics.

| Rule # | Total Blinks | Avg Squint Dur | Response Time | EAR Decline % | Fatigue Score | Classification | Action |
|---|---|---|---|---|---|---|---|
| 1 | < 30 | < 5 | Stable | < 5% | 0.00 - 0.20 | Minimal | None |
| 2 | 30-60 | 5-10 | Slight increase | 5-10% | 0.20 - 0.40 | Low | Monitor |
| 3 | 60-90 | 10-15 | Moderate increa | 10-15% | 0.40 - 0.60 | Moderate | Suggest break |
| 4 | 90-120 | 15-20 | Significant incre | 15-20% | 0.60 - 0.80 | High | Recommend break |
| 5 | > 120 | > 20 | Steep increase | > 20% | 0.80 - 1.00 | Severe | Stop test |

*Figure 12: Fatigue Level Classification Decision Table*

Fatigue Score Calculation: Fatigue = (0.30 × blink_factor) + (0.25 × squint_factor) + (0.25 × rt_trend_factor) + (0.20 × ear_decline_factor) Where each factor [0, 1] normalized by thresholds above.

## Refractive Error Classification Decision Table

Purpose: Classify type of refractive error based on diopter estimate and test patterns.

| Rule # | Diopter Mean | Diopter Std | Astigmatism In | Snellen vs Tum | Classification | Recommendation |
|---|---|---|---|---|---|---|
| 1 | < -0.5 | < 0.5 | < 0.3 | < 0.2 | Mild Myopia | Monitor, consider glasses if progressing |
| 2 | -0.5 to -3.0 | < 0.8 | < 0.3 | < 0.2 | Moderate Myopia | Recommend optometrist visit |
| 3 | < -3.0 | < 1.0 | < 0.3 | < 0.2 | High Myopia | Urgent optometrist referral |
| 4 | 0.5 to 3.0 | < 0.8 | < 0.3 | < 0.2 | Hyperopia | Recommend optometrist visit |
| 5 | > 3.0 | < 1.0 | < 0.3 | < 0.2 | High Hyperopia | Urgent optometrist referral |
| 6 | Any | Any | > 0.4 | > 0.3 | Astigmatism Likely | Recommend cylinder measurement |
| 7 | -0.5 to 0.5 | < 0.5 | < 0.3 | < 0.2 | Normal/Emmetropic | No action needed |
| 8 | Any | > 1.5 | Any | > 0.5 | Uncertain/Variable | Repeat test or consult professional |

*Figure 13: Refractive Error Classification Decision Table*

Indicator Calculations: - Astigmatism Indicator: Standard deviation of sector clarity scores in astigmatism dial test - Snellen vs Tumbling E Difference: |logMAR_snellen - logMAR_tumbling_e|

## Bayesian Model Selection Decision Table

Purpose: Select appropriate Bayesian threshold model parameters based on test type and user characteristics.

| Rule # | Test Type | User Age Group | Guess Rate | Slope Parameter | Initial Prior | **Rationale** |
|---|---|---|---|---|---|---|
| 1 | Snellen Acuity | Adult | 0.10 | 0.12 | Uniform | 10 letters, low guess chance |
| 2 | Snellen Acuity | Child (< 12) | 0.15 | 0.15 | Uniform | Higher guess rate for children |
| 3 | Tumbling E | Adult | 0.25 | 0.12 | Uniform | 4 directions, higher guess rate |
| 4 | Tumbling E | Child | 0.30 | 0.15 | Uniform | Children more likely to guess |
| 5 | Contrast Sensitivity | Any | 0.10 | 0.10 | Uniform | Single letter, harder to guess |
| 6 | Astigmatism Dial | Any | 0.08 | 0.08 | Uniform | Subjective judgment, minimal guessing |

*Figure 14: Bayesian Model Selection Decision Table*

Parameter Meanings: - Guess Rate: Probability of correct response by pure chance - Slope: Steepness of psychometric function (higher = more discriminative) - Prior: Initial belief distribution over threshold levels

## *Test Execution Flow Decision Table*

*Purpose: Determine next action during test execution based on current state.*

| Rule # | Face Detected | Glasses Detected | Baseline Ready | Trial Complete | All Tests Done | **Next Action** |
|---|---|---|---|---|---|---|
| 1 | No | - | - | - | - | Display "Position face in frame" |
| 2 | Yes | Yes | - | - | - | Display "Remove glasses" warning |
| 3 | Yes | No | No | - | - | Continue baseline calibration |
| 4 | Yes | No | Yes | No | No | Render next stimulus |
| 5 | Yes | No | Yes | Yes | No | Move to next trial |
| 6 | Yes | No | Yes | Yes (all trials) | No | Move to next test |
| 7 | Yes | No | Yes | Yes | Yes | Generate report |
| 8 | Lost mid-test | - | Yes | No | No | Pause, wait for re-detection |

*Figure 15: Test Execution Flow Decision Table*

## Response Validation Decision Table

Purpose: Validate user responses for each test type.

| Rule # | Test Type | Expected Response | User Input | Timing | **Validation Result** | **Action** |
|---|---|---|---|---|---|---|
| 1 | Snellen | "E" | "E" or "e" | $< 4$s | Valid (Correct) | Update Bayesian (success) |

*Figure 16: Response Validation Decision Table*

| Rule # | Test Type | Expected Response | User Input | Timing | **Validation Result** | **Action** |
|---|---|---|---|---|---|---|
| 2 | Snellen | "E" | "F" | $< 4$s | Valid (Incorrect) | Update Bayesian (failure) |
| 3 | Snellen | Any | No input | $> 4$s | Timeout | Treat as incorrect |
| 4 | Tumbling E | Arrow Up (↑) | Key: 82 | $< 4$s | Valid (Correct) | Update Bayesian (success) |
| 5 | Tumbling E | Arrow Right (→) | Key: 84 | $< 4$s | Valid (Incorrect) | Update Bayesian (failure) |
| 6 | Contrast | "C" | "C" or "c" | $< 4$s | Valid (Correct) | Update Bayesian (success) |
| 7 | Astigmatism | Multiple sectors | "3, 9" | No limit | Valid | Record sectors |
| 8 | Any | Any | ESC key | Any | Abort | Exit test, save partial results |

*Figure 17: Example of Response Validation Flow*

**Key Codes (OpenCV):** - Arrow Up: 82 - Arrow Down: 84 - Arrow Left: 81 - Arrow Right: 83 - Space: 32 - ESC: 27

## Calibration Quality Assessment Decision Table

**Purpose**: Assess quality of calibration and decide if recalibration is needed.

| Rule # | PPI Estimate | Distance Selection | Baseline Frames Collected | Face Stability (frames lost) | **Quality Rating** | **Action** |
|---|---|---|---|---|---|---|
| 1 | 80-120 | 40-80 cm | 60/60 | $< 5$ | Excellent | Proceed to tests |

*Figure 18: Calibration Quality Assessment Decision Table*

| Rule # | PPI Estimate | Distance Selection | Baseline Frames Collected | Face Stability (frames lost) | **Quality Rating** | **Action** |
|---|---|---|---|---|---|---|
| 2 | 70-130 | 40-80 cm | 55-59/60 | 5-10 | Good | Proceed with warning |
| 3 | 60-140 | 30-90 cm | 50-54/60 | 10-15 | Fair | Recommend recalibration |
| 4 | $< 60$ or $> 140$ | Any | $< 50/60$ | $> 15$ | Poor | Force recalibration |
| 5 | Any | $< 30$ or $> 90$ | Any | Any | Out of range | Display error, force recalibration |

*Figure 19:* Calibration Process Flow

**Calibration Quality Score**: Quality = w1 × ppi_score + w2 × distance_score + w3 × baseline_completeness + w4 × stabilityWhere: - ppi_score = 1 - |ppi - 96| / 50 - distance_score = 1 if 40-80cm, else 0.5 - baseline_completeness = frames_collected / 60 - stability_score = 1 - (frames_lost / total_frames) - Weights: w1=0.25, w2=0.20, w3=0.30, w4=0.25

## 3.2 Component Interactions and Collaborations

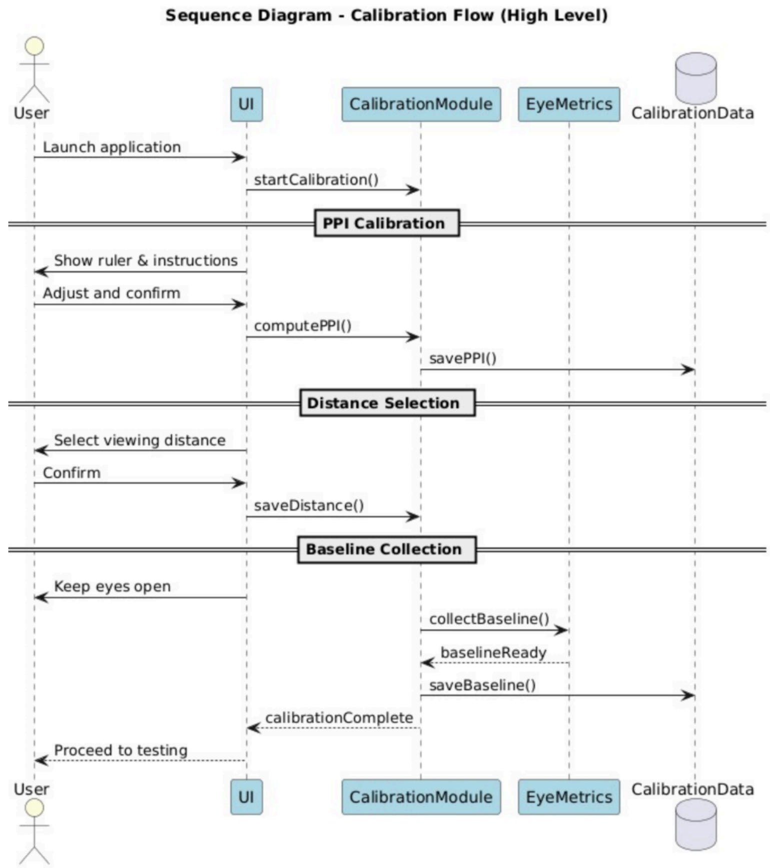


*Figure 20: Sequence Diagram Calibration flow*

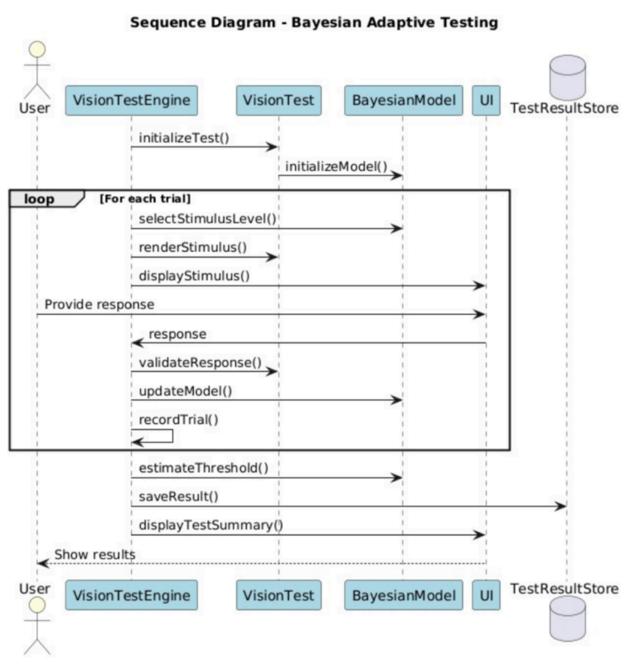


*Figure 20: Sequence Diagram Bayesian testing*

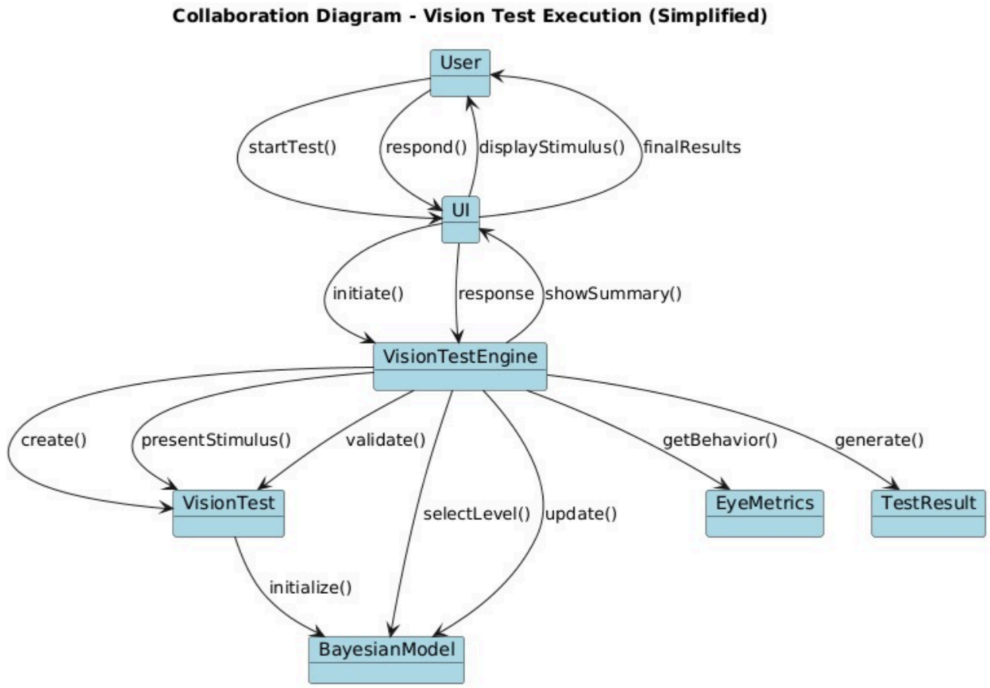


*Figure 21: Design Level Sequence Diagram*

This collaboration diagram depicts the object-level interactions involved in executing a single vision test within the Visuar system. It illustrates how the user initiates the test through the UI, which delegates control to the Vision Test Engine acting as the central coordinator. The Vision Test Engine manages the lifecycle of the test by instantiating the appropriate Vision Test module, initializing the Bayesian Model, and orchestrating the adaptive testing loop.

During execution, the Vision Test Engine requests stimulus generation, validates user responses, retrieves behavioral metrics from the EyeMetrics module, and updates the Bayesian Model based on trial outcomes. The Bayesian Model supports adaptive stimulus selection and threshold estimation by maintaining and updating probabilistic beliefs. Once all trials are completed, the Vision Test Engine aggregates trial data and generates a TestResult object, which is then presented to the user via the UI. Overall, the diagram highlights clear responsibility separation, message-based coordination, and the adaptive feedback loop that underpins Visuar's uncertainty-aware testing approach.

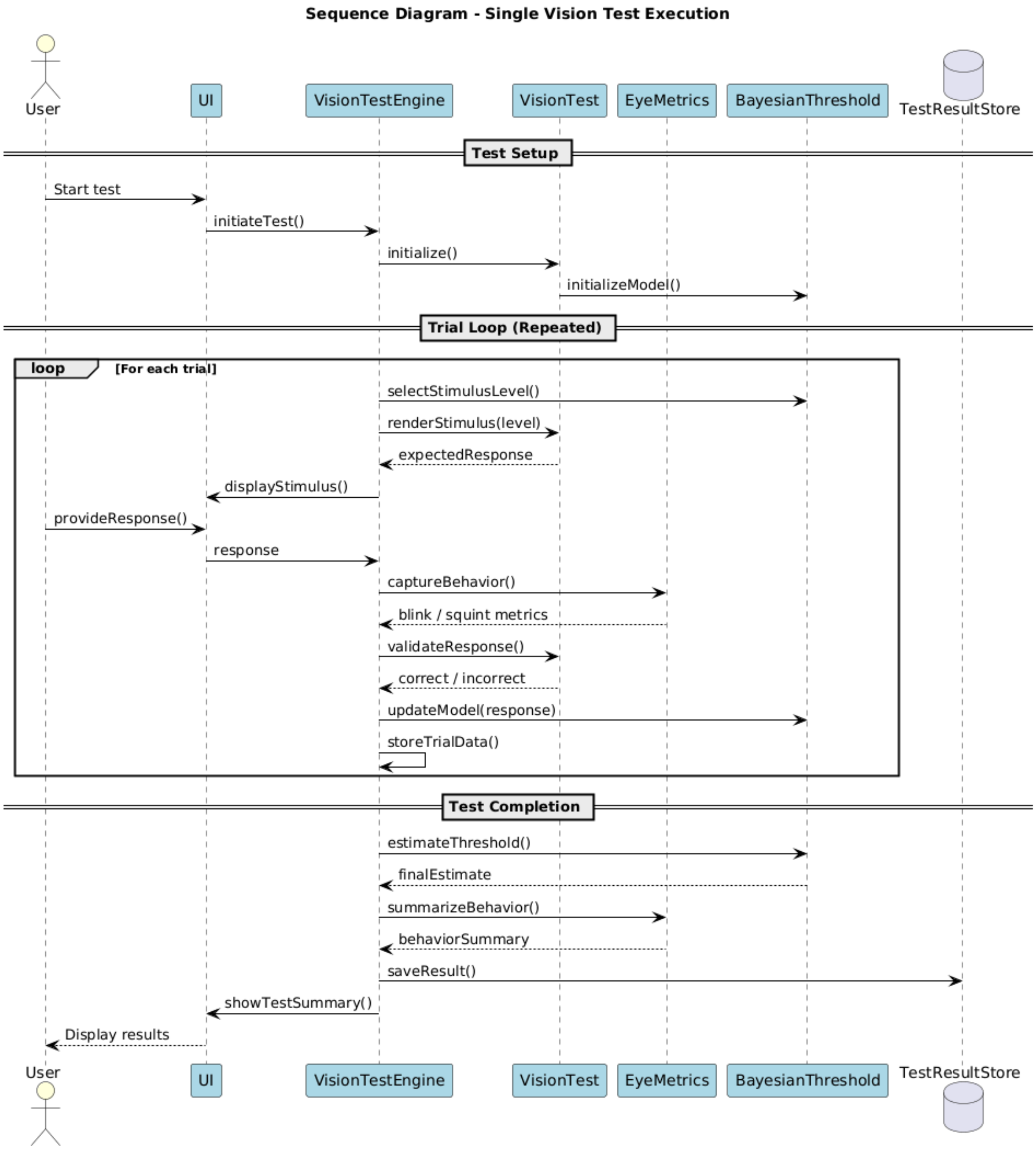


*Figure 22: Calibration and Test Engine Interaction*

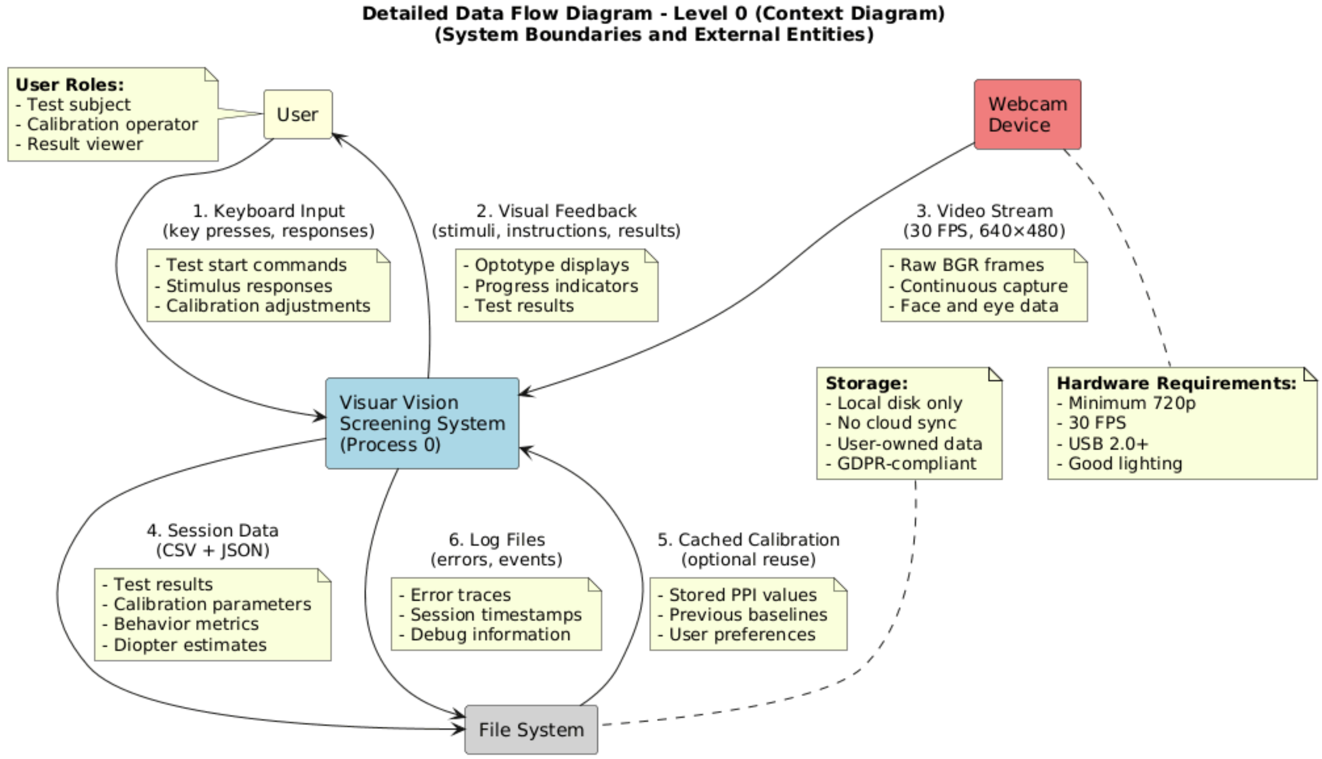


*Figure 23: Detailed Data Flow Diagram 0*

The primary external entity interacting with the system is the **User**, who may assume multiple roles, including test subject, calibration operator, and result viewer. User inputs are provided through the keyboard and include test commands, stimulus responses, and calibration adjustments. In response, the system delivers visual feedback such as optotype displays, instructional prompts, progress indicators, and final test results. This bidirectional interaction highlights the system's interactive nature and its reliance on explicit user engagement throughout calibration, testing, and result review phases.

The **Webcam Device** serves as a secondary external entity, supplying a continuous video stream to the Visuar system. This stream consists of raw RGB frames captured at approximately 30 frames per second with a standard resolution of 640×480 pixels. The video input enables face and eye data extraction for real-time behavior tracking during vision tests. All image processing and analysis are performed within the system boundary, ensuring that no raw or processed visual data is transmitted externally.

Persistent data storage is handled through the **Local File System**, which supports three distinct categories of stored information. First, **session data** is stored in CSV and JSON formats and includes test results, calibration parameters, behavioral metrics, and diopter estimates. Second, **cached calibration data** may be retained for optional reuse, allowing previously computed PPI values, baseline eye metrics, and user preferences to be loaded in subsequent sessions. Third, **log files** are maintained to record errors, session timestamps, and debugging information, supporting traceability and system maintenance.

The diagram also explicitly highlights key **system constraints and design assumptions**, including minimum hardware requirements such as camera resolution, frame rate, USB connectivity, and adequate lighting conditions. Notably, Visuar is designed to operate without any external network connections. All computation, data processing, and storage occur locally, ensuring compliance with privacy principles and minimizing exposure of sensitive user data.

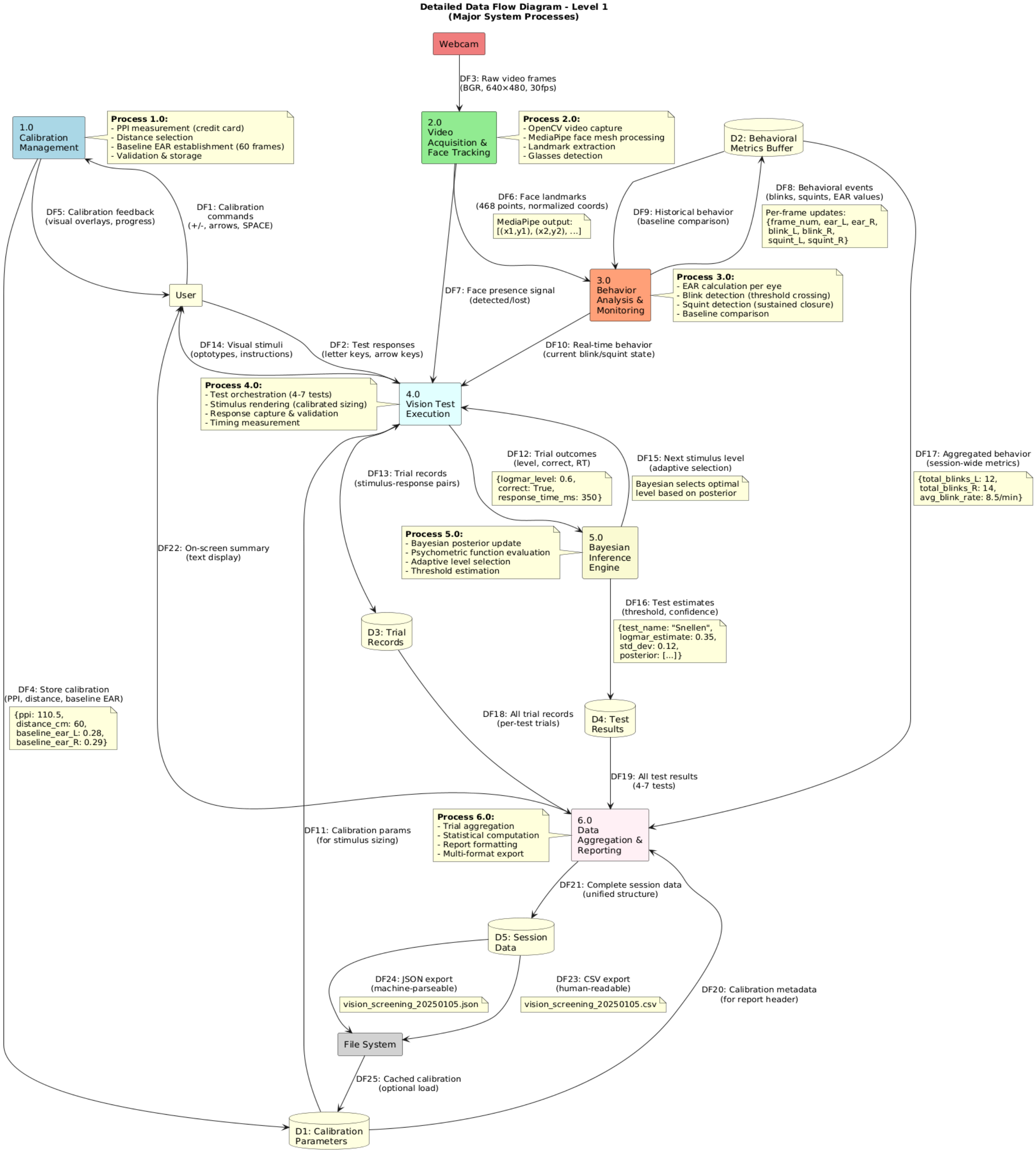


*Figure 24: data flow Diagram 1*

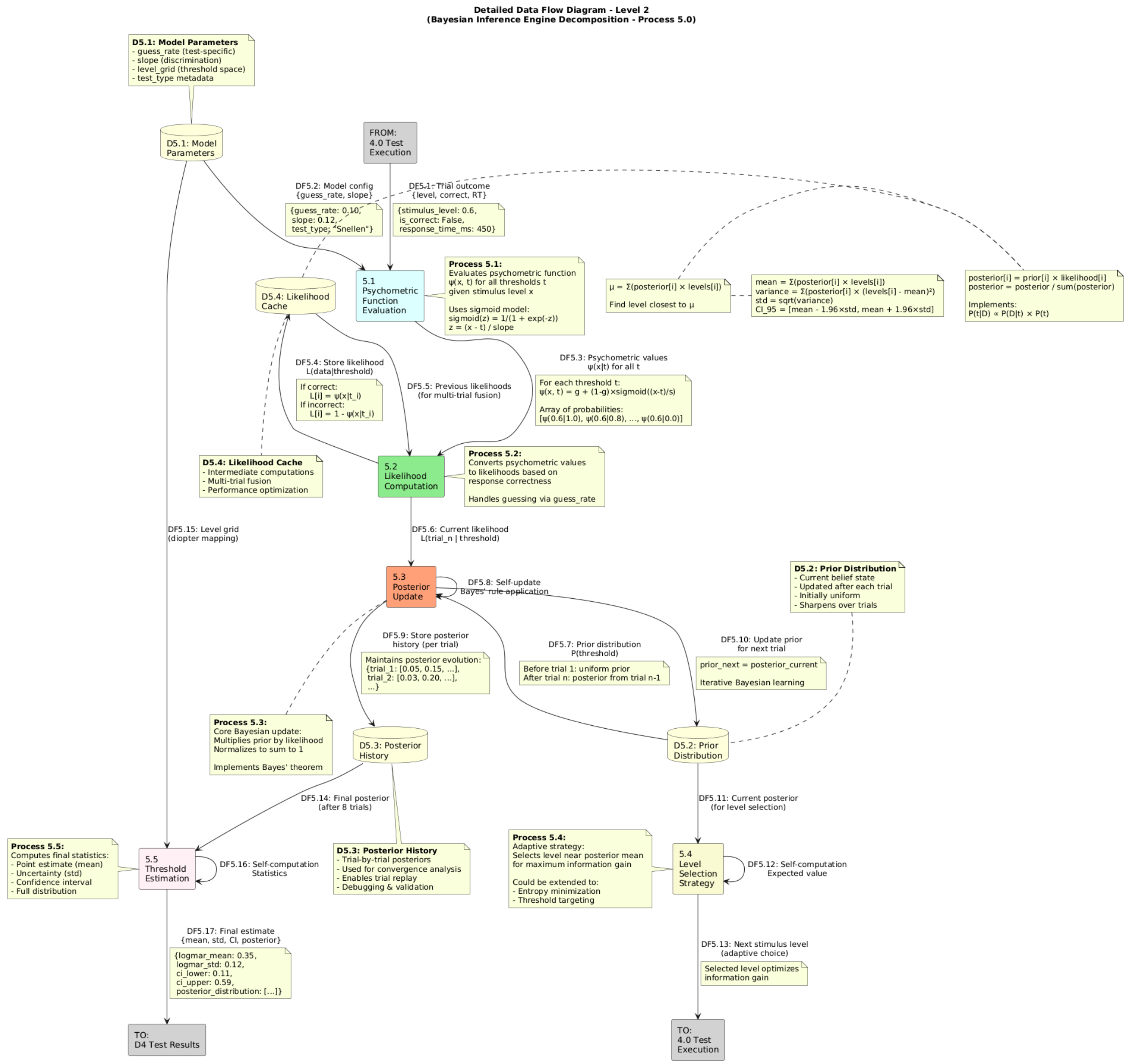


*Figure 25: Data Flow Diagram 2*

This diagram expands the internal logic of the probabilistic inference workflow, showing how individual trial outcomes from the vision test execution module are transformed into an uncertainty-aware visual estimate. The process begins with inputs from test execution and model parameters, which are used to evaluate psychometric responses and compute likelihood values for each possible threshold. These likelihoods are combined with the current prior distribution through a Bayesian posterior update, implementing iterative belief refinement across multiple trials. The diagram further shows how posterior distributions are stored and reused, enabling adaptive learning and convergence tracking over time. Based on the updated posterior, an adaptive level selection strategy determines the most informative stimulus level for subsequent trials, closing the feedback loop with the test execution process. After all trials are completed, the inference engine computes final summary statistics, including the estimated threshold and associated uncertainty, which are then passed to the test results output. Overall, this Level-2 DFD emphasizes Visuar's explainable and data-driven inference approach, highlighting clear separation between psychometric evaluation, Bayesian updating, adaptive decision-making, and final estimation, while avoiding black-box prediction models.

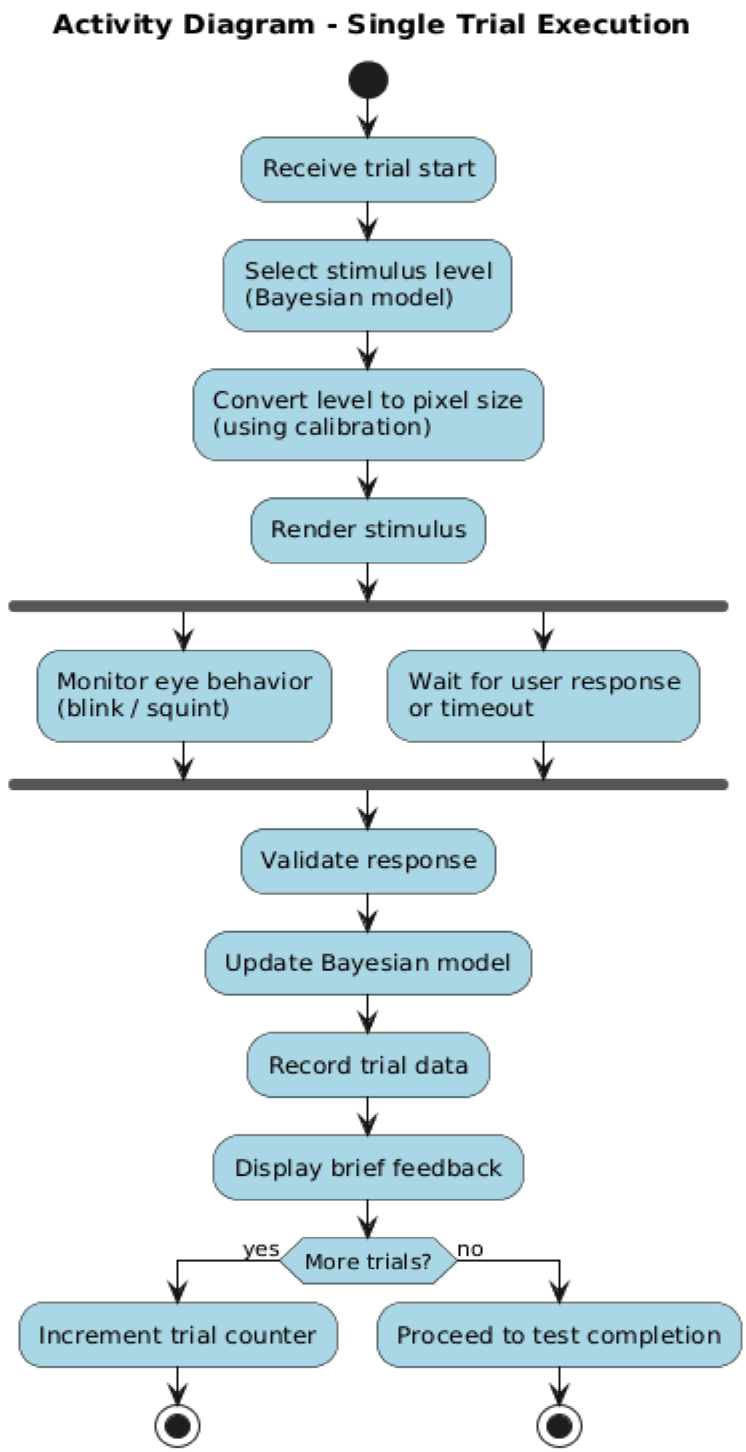


*Figure 26: Activity Diagram*

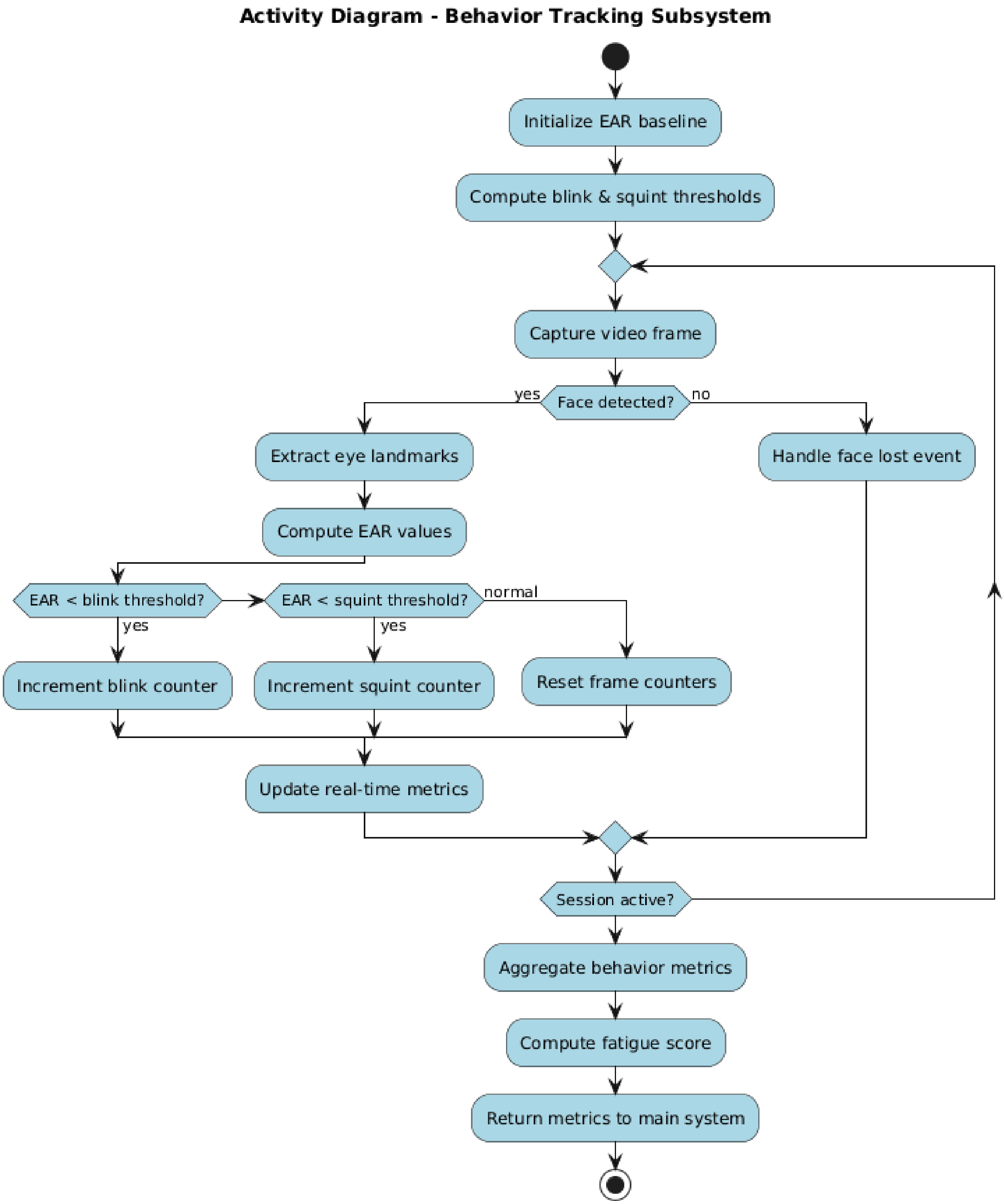


*Figure 27: Activity Diagram 1*

## 3.3 Design Reuse and Design Patterns

The Visuar system incorporates design reuse as a core principle to ensure consistency, reduce development effort, and simplify future expansion. During the current development phase, reuse is primarily applied at the **frontend interface level** and **basic backend service level**, which aligns with the project's prototype-focused progress.

On the frontend side, reusable UI components are employed across multiple screens, including the login page, dashboard layout, and data preview interfaces. Common elements such as input fields, buttons, typography styles, spacing rules, and color themes are shared throughout the application. This reuse ensures a consistent visual identity and user experience while minimizing redundant design and implementation work. Layout containers and navigation structures are also reused to maintain uniform interaction flow across different screens.

- **Strategy Pattern**: Test-specific probabilistic models
- **Pipeline Architecture**: Calibration → Tests → Fusion
- **Model-View Separation**: UI separated from inference logic

On the backend side, design reuse is applied through shared request-handling logic and validation routines. Basic API handlers for user authentication, request validation, and response formatting are implemented in a reusable manner, allowing the same logic to be extended or adapted as new features are added. Sample dataset handling follows a common data-access approach, enabling consistent data retrieval and display during the prototype demonstration.

From a design pattern perspective, Visuar follows a **modular and layered architectural approach**. The system separates concerns by clearly distinguishing between:

- **Presentation Layer** (frontend UI and user interaction),
- **Application Layer** (backend logic and request handling),
- **Data Layer** (sample datasets and data exchange).

This structure closely resembles the **Model–View–Controller (MVC)** pattern in practice, even if not implemented as a strict framework-level MVC. The frontend acts as the View, backend services function as the Controller, and datasets represent the Model. This separation improves readability, maintainability, and ease of testing.

Additionally, the use of a layered architecture supports scalability by allowing individual components to be modified or replaced independently. For example, the current sample dataset module can later be replaced with a persistent database or AI-generated results without impacting the frontend design. This reuse-oriented design strategy ensures that Visuar can evolve from a prototype into a more advanced system with minimal structural changes.

## 3.4 Technology Architecture

The technology architecture of Visuar defines the high-level technical infrastructure required to support the application, its processing logic, and user interaction. At the current stage of development, the system is designed as a lightweight, modular software solution that can operate on standard consumer hardware without the need for specialized medical equipment. The architecture emphasizes accessibility, scalability, and ease of future integration with advanced AI components.

Visuar follows a **client–server architectural model**, where the frontend handles user interaction and presentation logic, while the backend manages authentication, data exchange, and processing coordination. The frontend prototype demonstrates core interface elements such as the login screen, layout structure, navigation flow, and color themes. The backend provides basic API endpoints to support user authentication, data submission, and retrieval of sample dataset records used for demonstration purposes.

The system is intended to run primarily on desktop or web-based platforms and is compatible with commonly used operating systems. Communication between frontend and backend components is handled through standard HTTP-based APIs. During the prototype phase, local hosting is used for development and testing; however, the architecture is designed to support cloud-based deployment in later stages. The modular design ensures that future components—such as computer vision processing, machine-learning models, and persistent databases—can be integrated without major architectural changes.[1]

Connectivity requirements are minimal, allowing the system to function in environments with basic internet access. Core frontend demonstrations can operate locally, while backend interaction requires network connectivity. Security considerations, including controlled access to backend services and safe handling of user credentials, are incorporated at a foundational level. Overall, the technology architecture provides a stable and extensible foundation for evolving Visuar from a prototype into a fully functional AI-driven eye wellness system.

| Layer / Component | Technology Used | Purpose |
|---|---|---|
| **Presentation Layer** | HTML / CSS / JavaScript (Frontend Prototype) | Provides user interface, login page, layouts, navigation, and visual design |
| **Application Layer** | Python-based Backend (e.g., FastAPI / Flask) | Handles authentication, API routing, and basic application logic |
| **Data Handling** | Sample Dataset (Local / Static) | Used to demonstrate data flow and frontend-backend integration |
| **Communication** | HTTP / REST APIs | Enables interaction between frontend and backend components |
| **Hosting (Prototype)** | Localhost / Development Server | Supports local testing and demonstration of the system |
| **Platform Support** | Desktop / Web Environment | Ensures compatibility with commonly available systems |

# 4. Screenshots/Prototype

## 4.1 Workflow

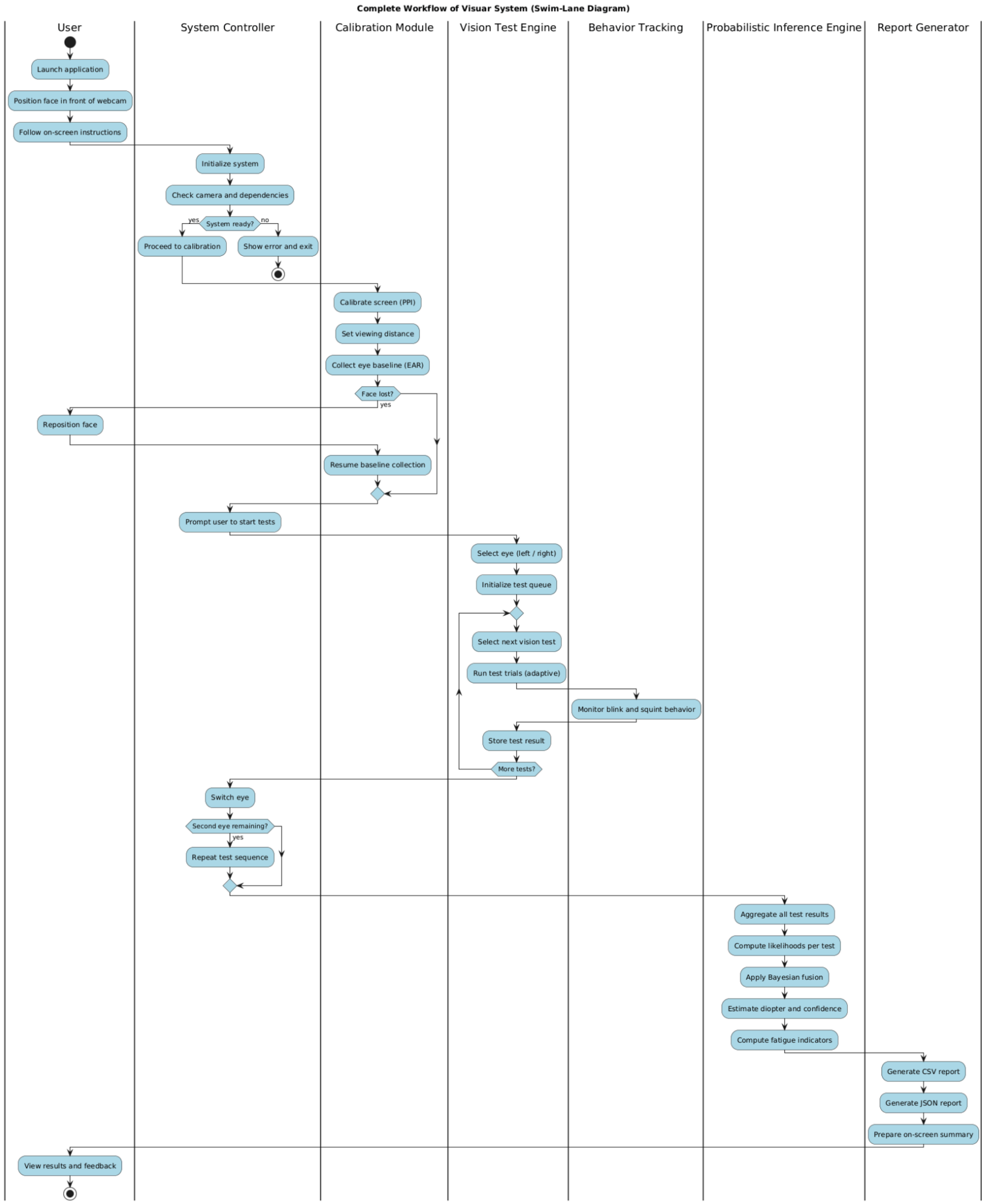


*Figure 28: Swim Lane Workflow*

*Visuar*

This swim-lane diagram illustrates the complete end-to-end workflow of the Visuar system. It highlights user interaction, system coordination, calibration, vision test execution, continuous behavior tracking, probabilistic inference using Bayesian fusion, and automated report generation. The diagram emphasizes separation of responsibilities across system modules while maintaining a clear execution flow.

## 4.2 Screens

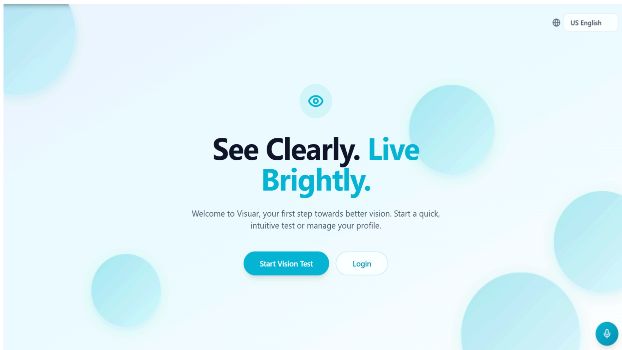


*Figure 28: screen*

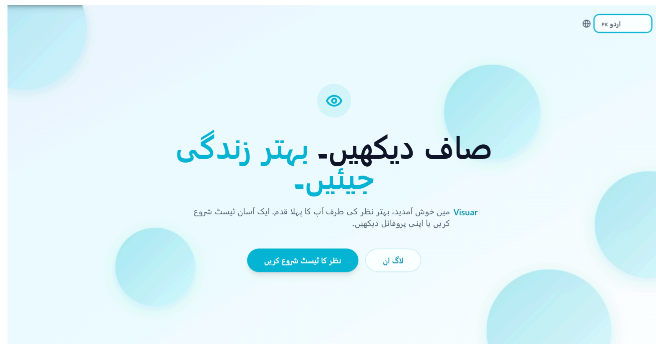


*Figure 29: screen*

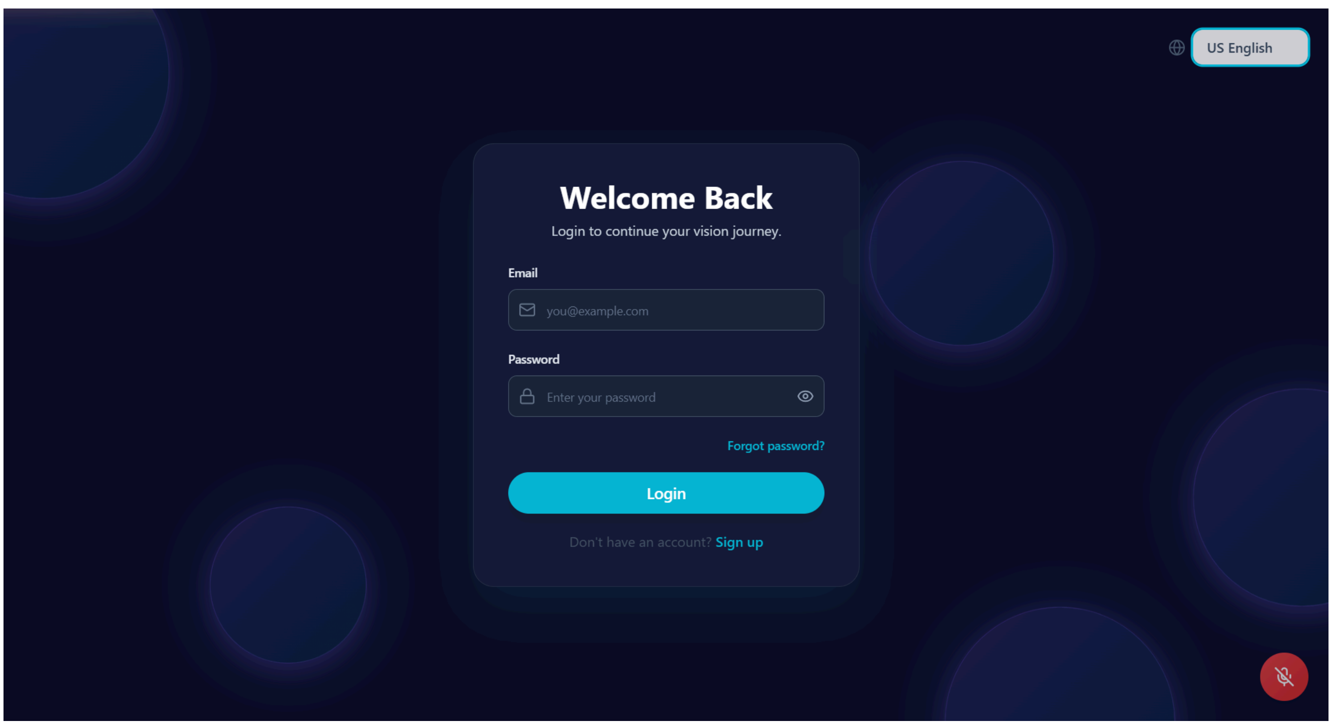


*Figure 30: login-dark mode*

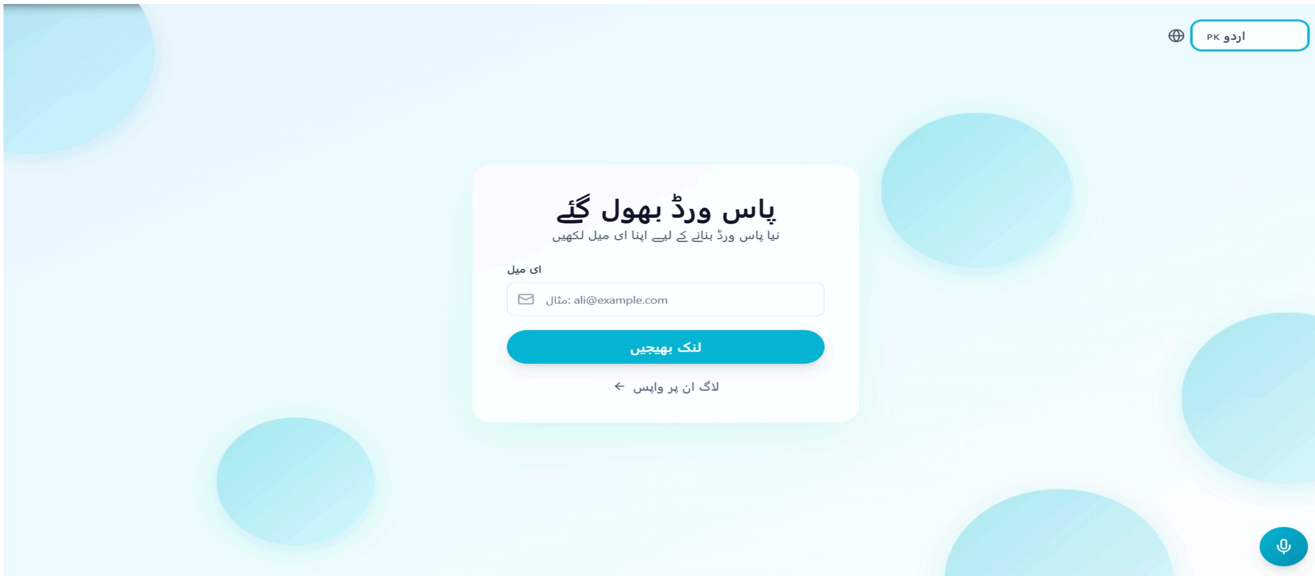


*Figure 31: reset password*

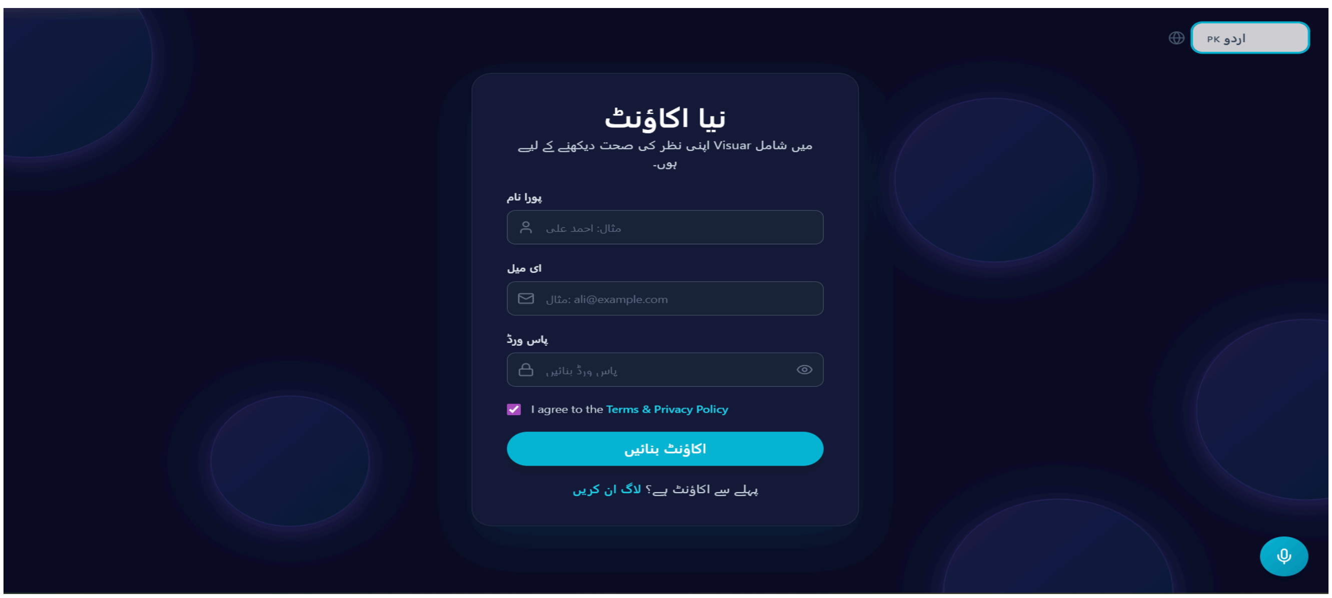


*Figure 32: Signup Screen-dark mode*

*Visuar*

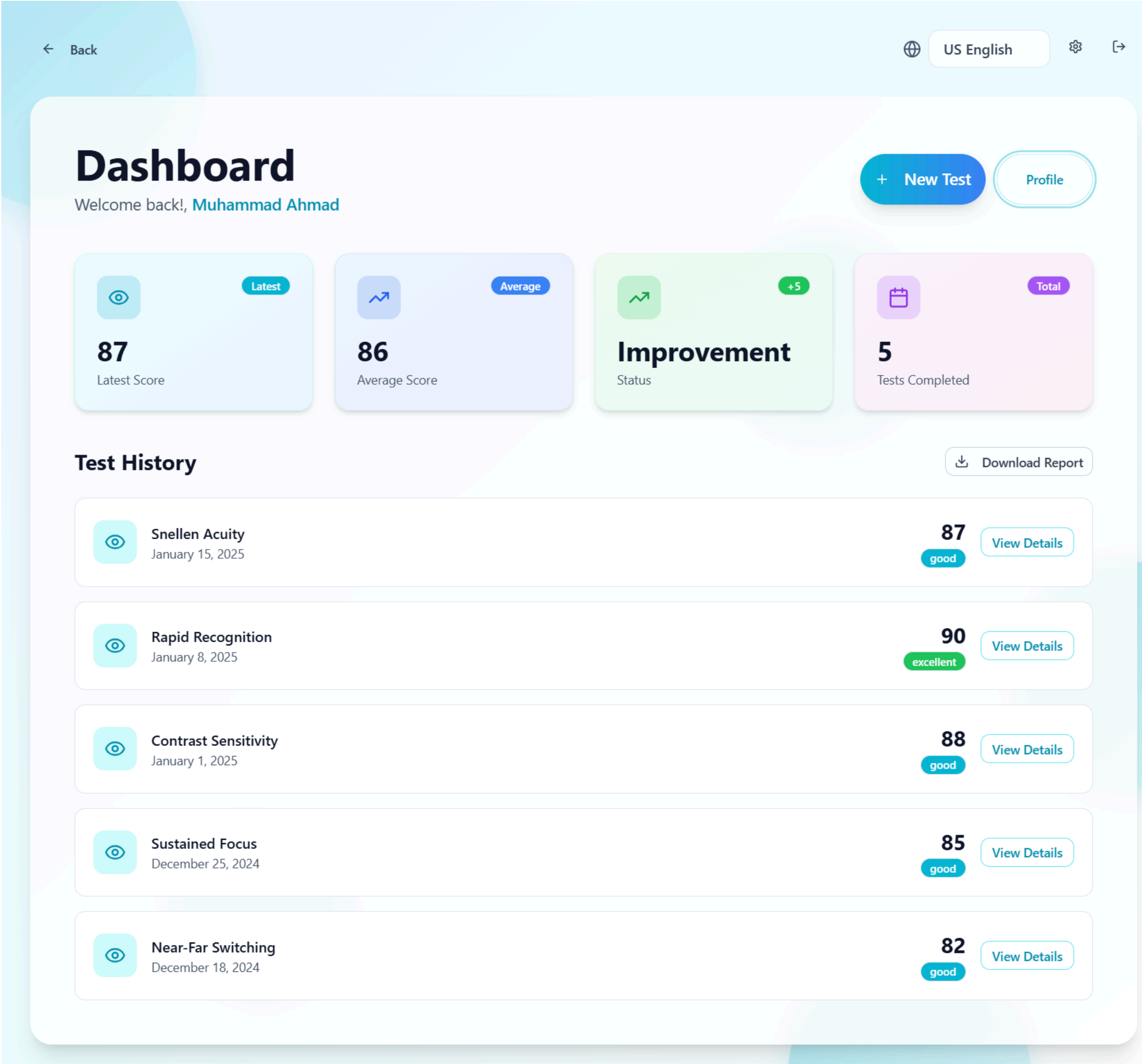


*Figure 33: dashboard*


*Figure 34: Dashboard-darkmode*

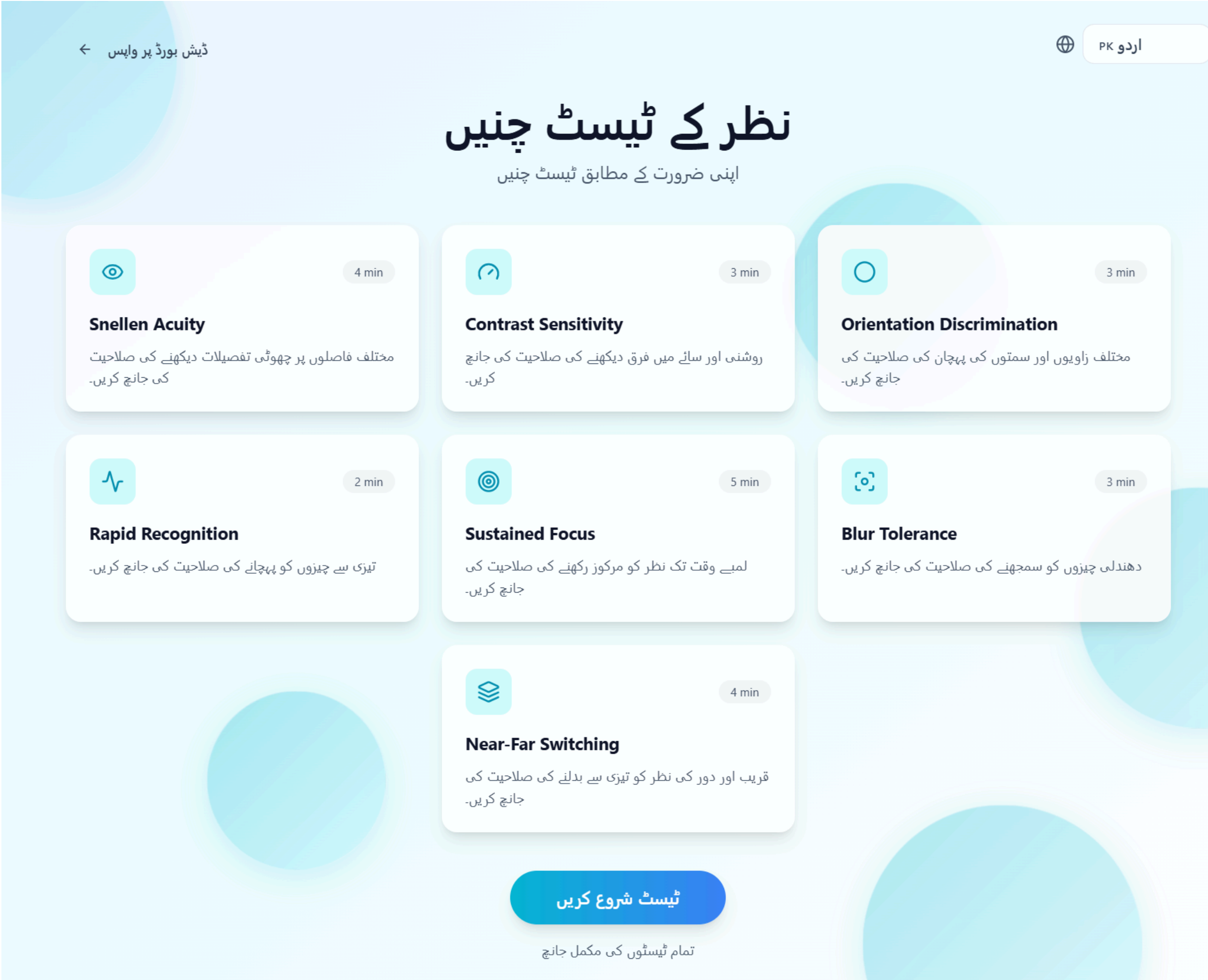


*Figure 35: test screen*


*Figure 36: test start permissions-darkmode*

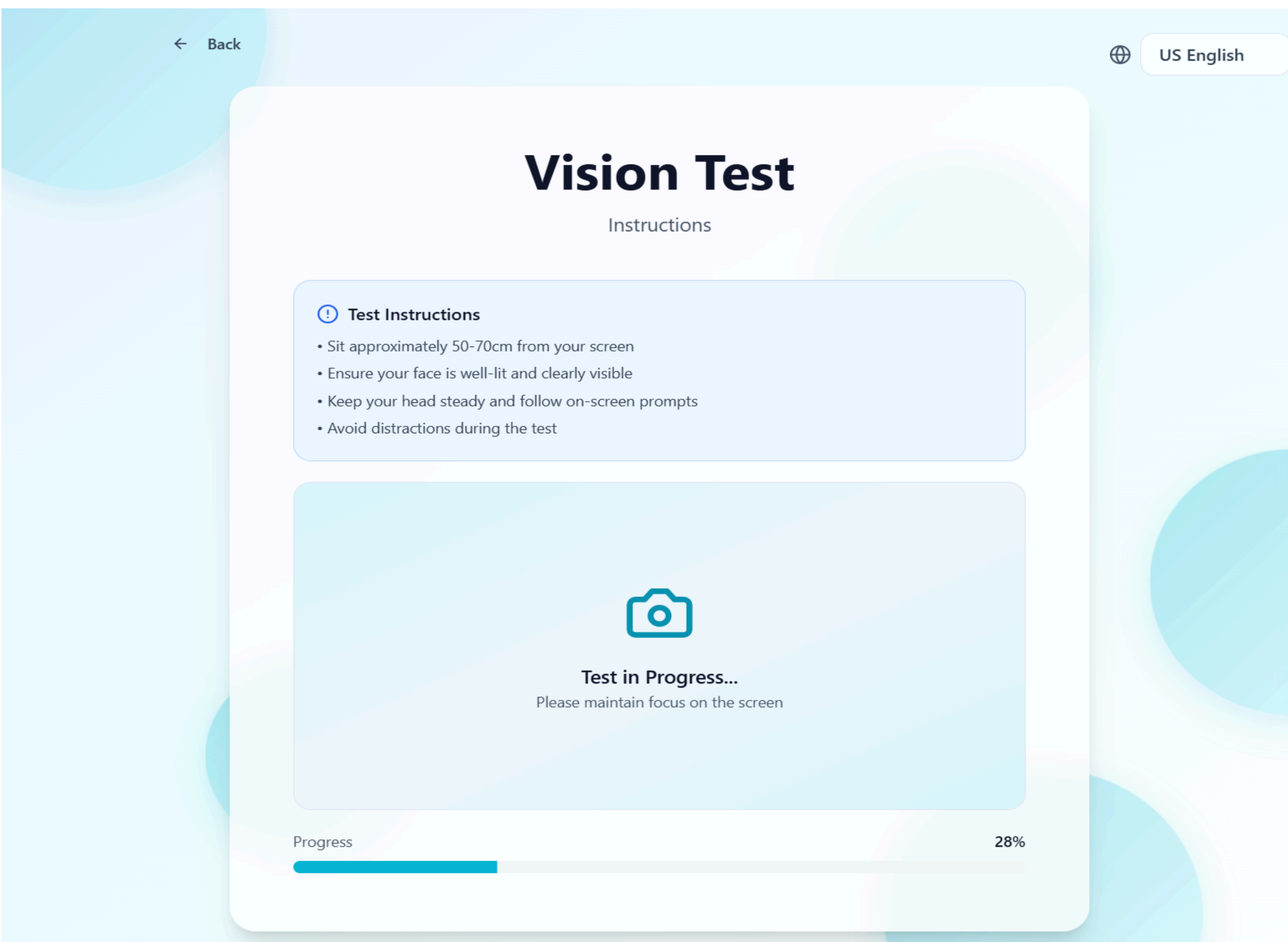


*Figure 37: Test start*

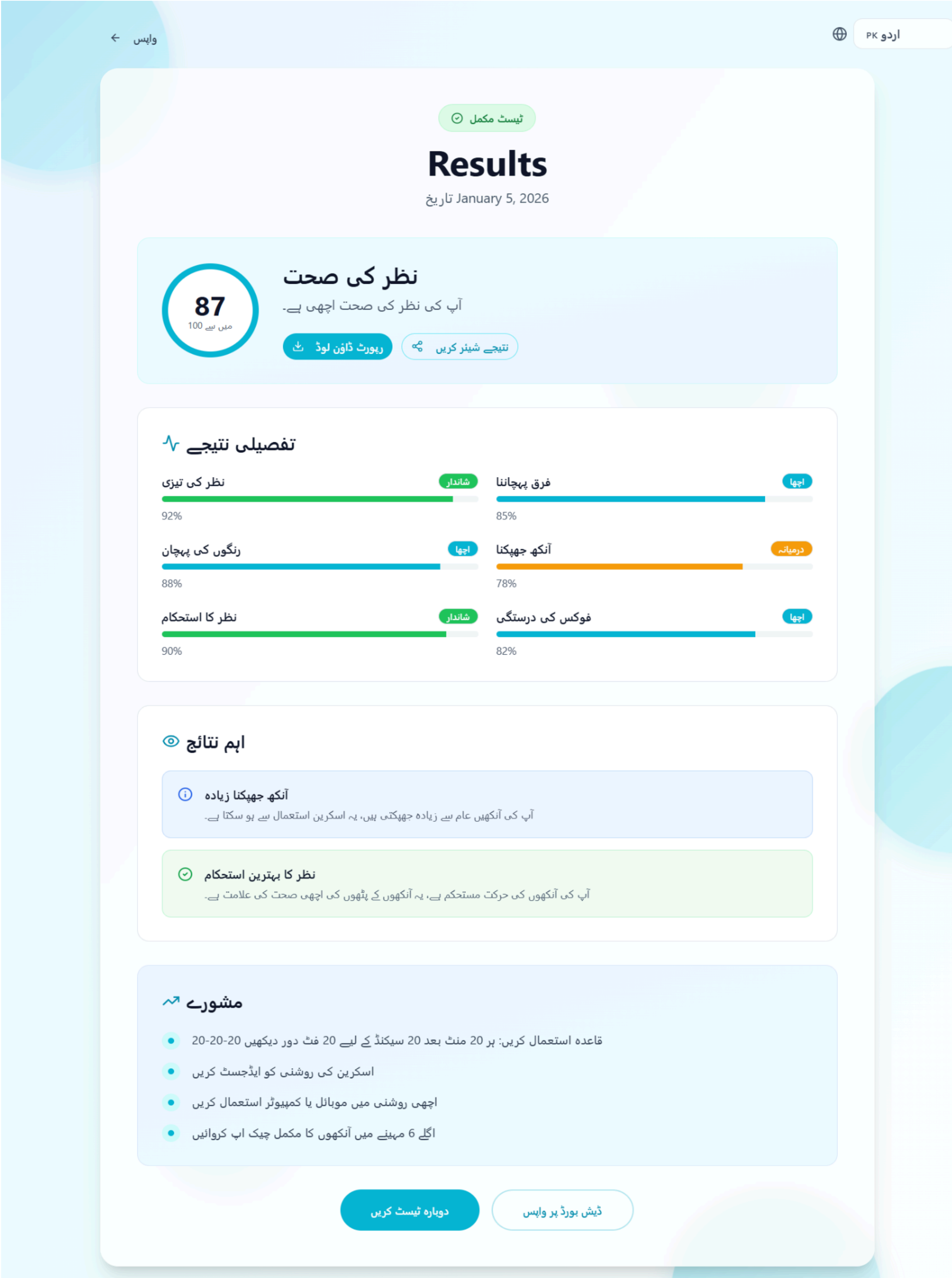


Figure 38: Results

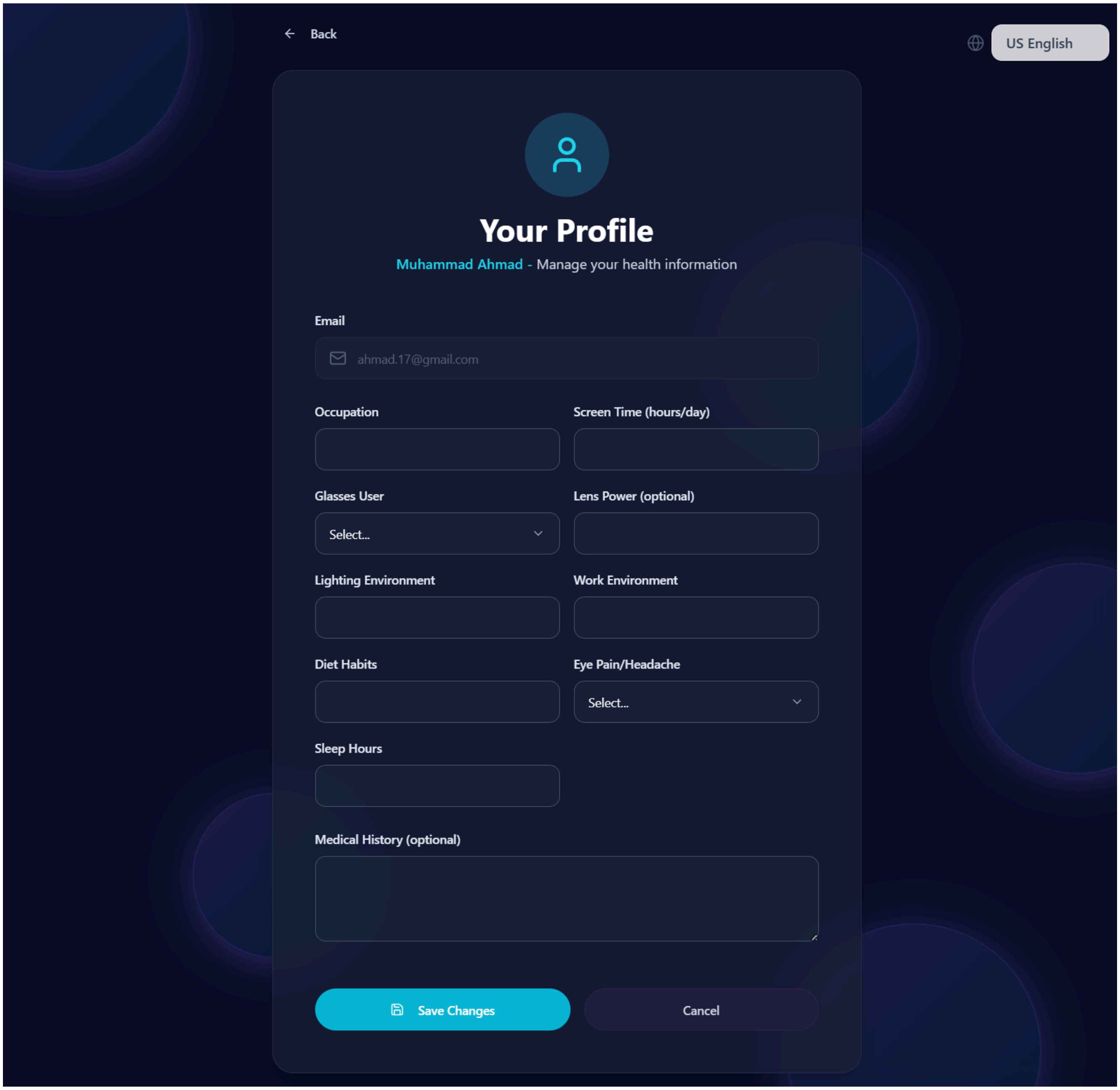


*Figure 39: profile-darkmode*

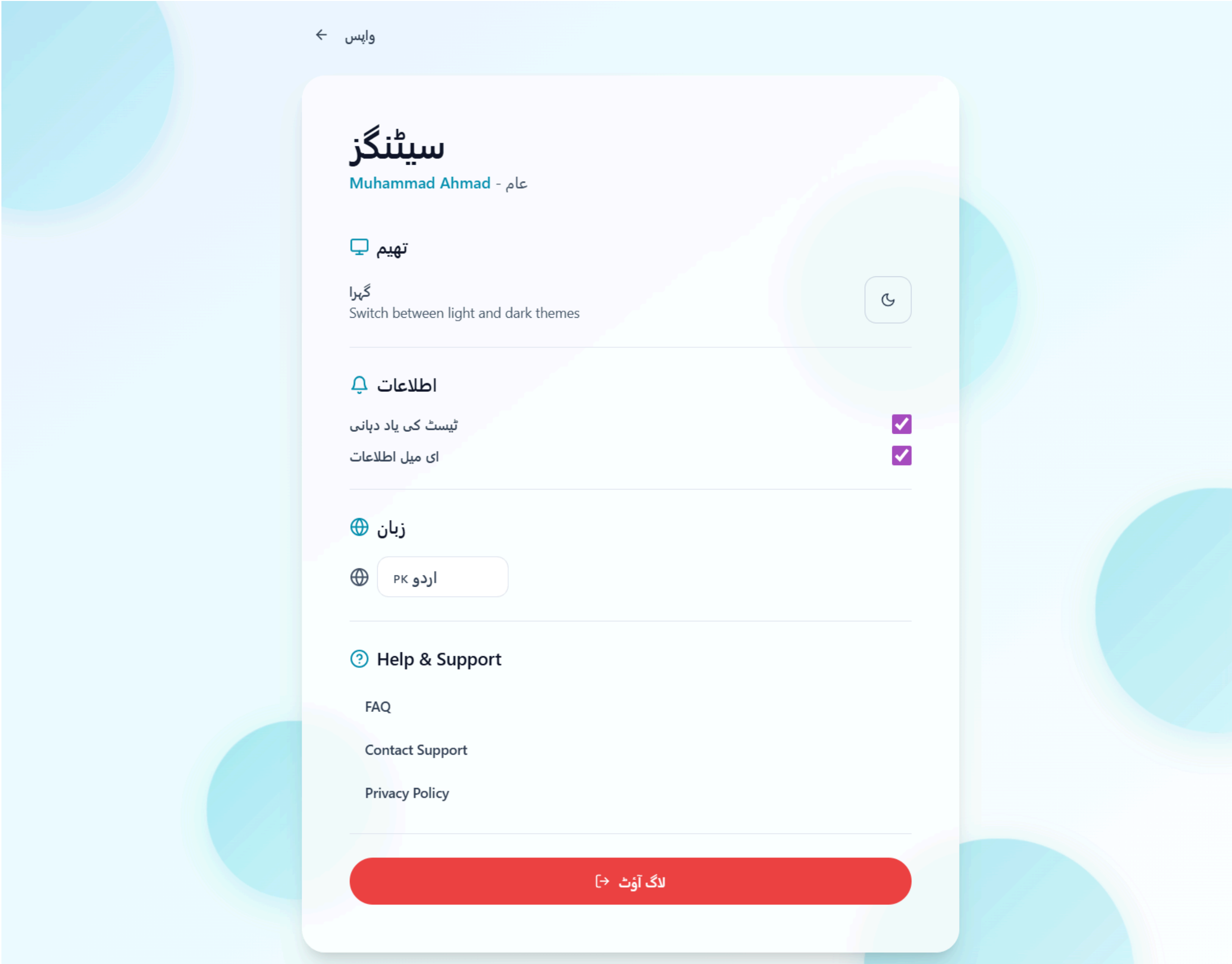


*Figure 40: settings*

## 4.3 Additional Information

During the current phase of development, the primary focus of the Visuar project has been on validating the system's visual design, user interaction flow, and basic frontend–backend connectivity rather than delivering a fully functional AI-based vision screening solution. This phased approach was intentionally adopted to ensure that foundational design decisions, usability aspects, and architectural feasibility are thoroughly evaluated before integrating complex computer vision and machine-learning components.

The implemented prototype demonstrates key frontend elements, including a functional login interface, consistent layout structure, navigation flow between screens, and selected color themes aligned with accessibility and readability guidelines. These elements were presented during the project demonstration to showcase the overall look and feel of the application and to gather early feedback regarding user experience and interface clarity.

In addition to the frontend prototype, a basic backend setup was also demonstrated to illustrate system connectivity and data flow. The backend supports essential operations such as handling user input, managing authentication requests, and exchanging sample dataset records with the frontend. The use of sample data at this stage allows verification of integration logic and data presentation without introducing unnecessary complexity or performance overhead.

This incremental development strategy enables early identification of usability issues, layout inconsistencies, and integration challenges, which can be addressed prior to the implementation of advanced vision processing and AI modules. The current prototype serves as a proof of concept, confirming that the system architecture, interaction design, and component communication are viable and ready for subsequent expansion in later project phases.

# 5. Other Design Details

Visuar is developed as a **research-oriented Final Year Project**, with the primary objective of investigating the feasibility of software-based, webcam-driven eye wellness screening in non-clinical environments. Unlike commercial or fully deployed healthcare systems, the design of Visuar emphasizes experimentation, validation of concepts, and architectural soundness rather than immediate clinical accuracy or large-scale deployment. This research focus allows the system to explore interaction design, visual behavior analysis, and early-stage integration of AI-supported workflows in a controlled academic setting.

A key design consideration in Visuar is **modularity**. The system is intentionally structured so that frontend, backend, and analytical components can be developed, tested, and refined independently. During the current phase, emphasis has been placed on frontend prototyping and basic backend connectivity, enabling early evaluation of usability, layout consistency, and system flow. This modular approach ensures that future integration of advanced computer vision or machine-learning modules can be achieved without requiring major changes to the existing system structure.

**Privacy and ethical considerations** play an important role in the design of Visuar. Since the system deals with user interaction and visual data, design decisions have been made to minimize data exposure during the prototype phase. Only essential user information is handled, and no long-term storage of sensitive visual data is performed at this stage. This approach aligns with ethical research practices and ensures that the system remains compliant with academic guidelines for user data handling.[13][14]

From a **user experience (UX) perspective**, Visuar prioritizes simplicity, clarity, and accessibility. Design choices such as readable font sizes, adequate color contrast, and intuitive navigation are incorporated to support users with varying levels of technical proficiency. Special attention is given to ensuring that the interface remains understandable for diverse user groups, including general users and individuals with limited digital experience. Feedback obtained during prototype demonstrations is intended to guide further refinement of interaction flow and visual presentation.

Overall, the current prototype of Visuar serves as a **foundation for future development and experimentation**. It establishes a validated base upon which advanced features—such as real-time vision analysis, AI-based inference, and enhanced reporting—can be incrementally added in later phases. These design decisions ensure that the project remains realistic, ethically responsible, and aligned with the objectives of a research-based academic project.

# 6. Revised Project Plan

## 6.1 Overview of Revised Plan

This section presents the **updated project plan** for Visuar – Smart Eye Testing and Wellness Guidance, reflecting the **actual progress achieved so far**, lessons learned during prototyping, and refinements made after Phase-I evaluation.

The revised plan ensures that:

- The project remains **achievable within the BS Final Year timeline**
- Technical risks are **reduced through phased integration**
- Core AI-based vision screening objectives are **prioritized over optional enhancements**
- Documentation, testing, and validation receive **adequate time allocation**

The project continues to follow a **phased, milestone-driven approach**, where each phase produces a tangible and verifiable outcome.

## 6.2 Phase-wise Progress Status

| Phase | Planned Activities | Current Status |
|---|---|---|
| **Phase 1 – Requirement Analysis & Research** | Literature review, problem definition, SRS preparation | Completed |
| **Phase 2 – System Design (SDS)** | Architecture design, component modeling, data flow, decision tables | Completed |
| **Phase 3 – Frontend Prototype** | UI layout, navigation, interaction flow | Completed |
| **Phase 4 – Backend & API Setup** | FastAPI setup, authentication, data handling | In Progress |
| **Phase 5 – Vision Processing Modules** | Eye tracking, blink detection, gaze estimation | In Progress |
| **Phase 6 – AI Inference & Fusion Logic** | Probabilistic modeling, fatigue scoring | Planned |
| **Phase 7 – Report Generation** | PDF reports, summaries, export | Planned |
| **Phase 8 – Testing & Validation** | Functional testing, IV&V | Planned |
| **Phase 9 – Final Documentation & Defense** | Report finalization, demo prep | Planned |

This revised breakdown reflects a **shift from UI-first development to AI-first stabilization**, which is necessary due to webcam variability and real-time performance constraints.

## 6.3 Revised Timeline Explanation

The revised timeline emphasizes **incremental AI integration** instead of a monolithic implementation. Vision modules are developed and validated **independently** before being fused into the final inference engine.

Key refinements include:

- Early isolation of **calibration and eye-tracking instability issues**
- Delayed integration of **LLM-based wellness interpretation** until core metrics are reliable
- Explicit time reserved for **accuracy tuning and behavioral noise handling**

This approach minimizes late-stage failures and supports meaningful evaluation during defense.

## 6.4 Risk Handling & Contingency Planning

| Risk | Mitigation Strategy |
|---|---|
| *Webcam quality inconsistency* | *Adaptive thresholds, confidence weighting* |
| *Low FPS on weak systems* | *Frame skipping & lightweight models* |
| *User movement noise* | *Behavioral confidence scoring* |
| *Time constraints* | *Feature prioritization over feature count* |
| *Over-complex AI models* | *Use of probabilistic + rule-based fusion* |

*These strategies ensure that VISUAR remains **robust, explainable, and defensible** as a BS-level project.*

# 6.5 Gantt Chart

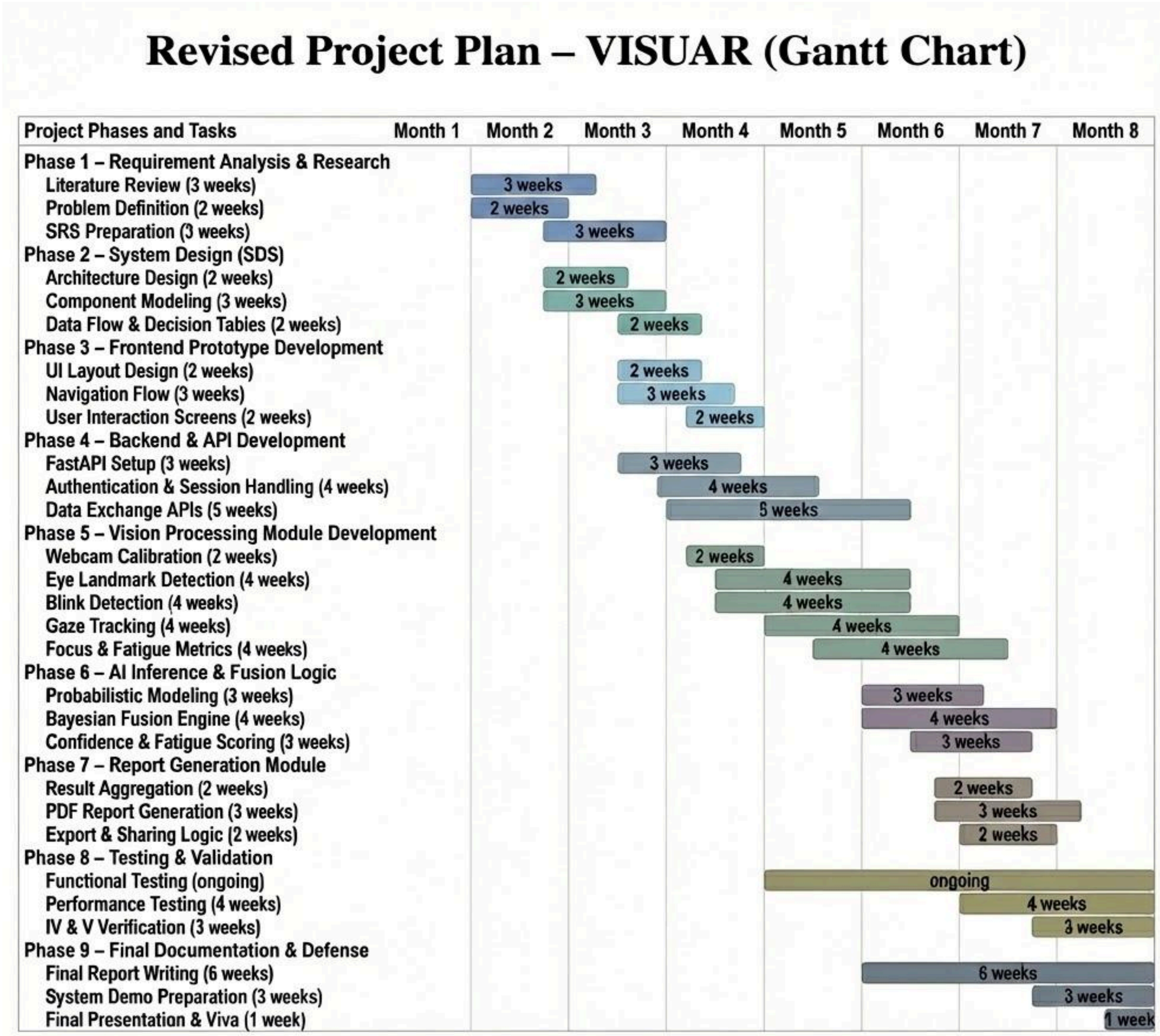


*Figure 41: Gantt Chart*

# 7. References


*[1] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000. [Online]. Available: https://opencv.org/*

*[2] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv preprint arXiv:1906.08172, 2019. [Online]. Available: https://arxiv.org/abs/1906.08172*

*[3] K. Holmqvist, M. Nyström, R. Andersson, R. Dewhurst, H. Jarodzka, and J. van de Weijer, Eye Tracking: A Comprehensive Guide to Methods and Measures. Oxford, U.K.: Oxford University Press, 2011.*

*[4] A. T. Duchowski, Eye Tracking Methodology: Theory and Practice, 3rd ed. Cham, Switzerland: Springer, 2017.*

*[5] A. B. Watson and D. G. Pelli, "QUEST: A Bayesian adaptive psychometric method," Perception & Psychophysics, vol. 33, no. 2, pp. 113–120, 1983.*

*[6] N. Prins and F. A. A. Kingdom, "Applying the model-comparison approach to test adaptive psychophysical methods," Journal of Vision, vol. 18, no. 3, pp. 1–13, 2018.*

*[7] ISO 9241-11:2018, Ergonomics of Human-System Interaction – Usability: Definitions and Concepts. Geneva, Switzerland: International Organization for Standardization, 2018.*

*[8] J. E. Sheedy, J. R. Hayes, and J. Engle, "Is all asthenopia the same?" Optometry and Vision Science, vol. 80, no. 11, pp. 732–739, 2003.*

*[9] S. Tiwari et al., "Computer vision-based eye blink detection for fatigue analysis," International Journal of Computer Applications, vol. 176, no. 29, 2020.*

*[10] D. W. Hansen and Q. Ji, "In the eye of the beholder: A survey of models for eyes and gaze," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 32, no. 3, pp. 478–500, Mar. 2010. [Online]. Available: https://ieeexplore.ieee.org/document/5206697*

*[11] J. A. Stern, L. C. Walrath, and R. Goldstein, "The endogenous eyeblink," Psychophysiology, vol. 21, no. 1, pp. 22–33, Jan. 1984. [Online]. Available: https://onlinelibrary.wiley.com/doi/10.1111/j.1469-8986.1984.tb00600.x*

*[12] M. Rosenfield, "Computer vision syndrome: A review," Survey of Ophthalmology, vol. 56, no. 5, pp. 377–386, Sep.–Oct. 2011. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0039625711000168*

*[13] D. A. Norman, The Design of Everyday Things, Rev. ed. New York, NY, USA: Basic Books, 2013. [Online]. Available: https://www.basicbooks.com/titles/don-norman/the-design-of-everyday-things/9780465050659/*

*[14] World Health Organization, Ethics and Governance of Artificial Intelligence for Health. Geneva, Switzerland: World Health Organization, 2021. [Online]. Available: https://www.who.int/publications/i/item/9789240029200*

# Appendix A: Glossary

**AI (Artificial Intelligence)**

The capability of a computer system to perform tasks that normally require human intelligence, such as pattern recognition, decision-making, and adaptive behavior.

**Activity Diagram**

A UML diagram used to represent workflows and control flow within a system, showing actions, decisions, and parallel activities.

**Adaptive Testing**

A testing approach where stimulus difficulty or level is dynamically adjusted based on user performance to improve efficiency and accuracy.

**API (Application Programming Interface)**

A set of rules and endpoints that allow software components to communicate with each other.

**Astigmatism**

A refractive error caused by irregular curvature of the cornea or lens, resulting in distorted or blurred vision.

**Astigmatism Dial Test**

A vision test that evaluates directional clarity differences to identify potential astigmatic patterns.

**Baseline**

Initial reference measurements collected during calibration, used for comparison during testing (e.g., baseline EAR).

**Bayesian Fusion**

The process of combining multiple probabilistic estimates into a single posterior distribution using Bayesian inference.

**Bayesian Inference**

A probabilistic reasoning method that updates beliefs based on prior knowledge and observed evidence using Bayes' theorem.

**Bayesian Model**

A statistical model that represents uncertainty using probability distributions rather than fixed values.

**Behavior Metrics**

Quantitative indicators of user behavior such as blink count, squint count, response time, and gaze stability.

**Blink**

A complete closure of the eyelids, typically detected using eye aspect ratio thresholds.

**Blink Rate**

The number of blinks per unit time, often used as an indicator of visual fatigue or strain.

**Calibration**

The process of adjusting system parameters (e.g., PPI, distance, baseline EAR) to ensure accurate and consistent measurements.

**Client–Server Architecture**

A system architecture where a client handles user interaction and a server manages processing and data exchange.

**Computer Vision**

A field of AI focused on enabling computers to interpret and analyze visual data from images or videos.

**Confidence Interval (CI)**

A statistical range that estimates the uncertainty around a computed value, typically expressed at a 95% confidence level.

**Confidence Score**

A normalized value representing the reliability of test results based on behavioral consistency and response quality.

**Contrast Sensitivity**

The ability to distinguish objects from their background based on differences in contrast.

**CSV (Comma-Separated Values)**

A simple file format used to store tabular data in plain text.

**DFD (Data Flow Diagram)**

A diagram that illustrates how data moves through a system, including processes, data stores, and external entities.

**Decision Table**

A tabular representation of conditional logic used to determine system actions based on input conditions.

**Deployment Diagram**

A UML diagram showing how software components are deployed on hardware and infrastructure.

**Diopter (D)**

A unit of measurement representing the refractive power of the eye or a lens.

**Digital Eye Strain**

Visual discomfort caused by prolonged use of digital devices, often associated with reduced blink rate and fatigue.

**EAR (Eye Aspect Ratio)**

A numerical measure of eye openness computed from eye landmarks, used for blink and squint detection.

**Eye Estimate**

The final inferred visual assessment for an eye, including diopter estimate, confidence, and fatigue indicators.

**Eye Tracking**

The process of measuring eye position and movement to analyze gaze behavior and visual activity.

**FaceMesh**

A MediaPipe model that detects and tracks facial landmarks in real time.

**Fatigue Score**

A computed value representing visual fatigue based on behavioral trends such as blink rate and response consistency.

**Functional Requirement**

A specification of what a system should do.

**Glasses Detection**

The process of identifying whether a user is wearing glasses during testing, as it may affect measurements.

**Hyperopia**

A refractive error where distant objects may be seen more clearly than near objects (farsightedness).

**IV & V (Independent Verification and Validation)**

A process used to evaluate whether a system meets its requirements and is built correctly.

**JSON (JavaScript Object Notation)**

A lightweight data-interchange format used for structured data storage and exchange.

**Likelihood Distribution**

A probability distribution representing how likely observed data is under different parameter values.

**logMAR**

Logarithm of the Minimum Angle of Resolution; a standardized scale for measuring visual acuity.

**MediaPipe**

An open-source framework developed by Google for building real-time computer vision pipelines.

**Myopia**

A refractive error where near objects are seen clearly but distant objects appear blurred (nearsightedness).

**Non-Clinical Screening**

An assessment intended for awareness and guidance rather than medical diagnosis or treatment.

**Optotype**

A standardized symbol or character used in vision tests (e.g., letters, Tumbling E).

**Posterior Distribution**

The updated probability distribution after combining prior belief with observed data.

**PPI (Pixels Per Inch)**

A measure of screen pixel density, used to convert visual angles into pixel sizes.

**Probabilistic Modeling**

An approach that represents uncertainty using probability distributions instead of deterministic values.

**Psychometric Function**

A function that models the relationship between stimulus intensity and response probability.

**Refractive Error**

A vision condition where light is not properly focused on the retina.

**Response Time (RT)**

The time elapsed between stimulus presentation and user response.

**Screening System**

A system designed to provide early indicators or awareness rather than definitive diagnosis.

**Sequence Diagram**

A UML diagram showing interactions between system components over time.

**Session**

A complete user interaction cycle from calibration to report generation.

**Snellen Test**

A visual acuity test using letters of decreasing size.

**Squint**

A partial closure of the eyelids, often associated with visual discomfort or difficulty focusing.

**State Diagram**

A UML diagram representing system states and transitions.

**Swim-Lane Diagram**

An activity diagram divided into lanes to show responsibility across components or actors.

**Test Result**

The structured outcome of a vision test, including performance metrics and behavior data.

**Threshold Estimation**

The process of estimating the stimulus level at which a user reliably perceives a visual target.

**Tumbling E Test**

A vision test where the user identifies the orientation of the letter “E”.

**Usability**

The extent to which a system can be used effectively, efficiently, and satisfactorily by users.

**Wellness Guidance**

Non-medical recommendations intended to improve eye comfort and reduce strain.

# Appendix B: IV & V Report

**(Independent verification & validation)**
**IV & V Resource**

| Name | Signature |
|---|---|

| S# | Defect Description | Origin Stage | Status | Fix Time | |
|---|---|---|---|---|---|
| | | | | Hours | Minutes |
| 1 | Login page alignment inconsistent on different screen sizes | UI Layout Design | Fixed | 0 | 20 |
| 2 | Color contrast on login screen not clearly visible on some displays | UI/UX Design Review | Fixed | 0 | 30 |
| 3 | Input fields on login page lack proper validation messages | Frontend Prototype | Fixed | 0 | 17 |
| 4 | Navigation between login and dashboard screens is not intuitive | Usability Evaluation | In Progress | 0 | 23 |
| 5 | Font size and spacing inconsistent across different frontend screens | Frontend Styling | Fixed | 0 | 55 |
| 6 | Error handling messages not shown when backend connection fails | System Integration | In Progress | 0 | 46 |

**Table 1: List of non-trivial defects**

This document has been adapted from the following:

1. Previous project templates at UCP
2. High-level Technical Design, Centers for Medicare & Medicaid Services. (www.cms.gov)